

2025년도
학사학위논문

온라인 여론조사의 조작 방지를 위한 블록체인 기반 시스템 설계 및 구현

Design and Implement of a
Blockchain-Based System to Prevent
Tampering in Online Public Opinion
Poll

2025년 06월 13일

순천향대학교 SW융합대학
컴퓨터소프트웨어공학과

백승훈

온라인 여론조사의 조작 방지를 위한
블록체인 기반 시스템 설계 및 구현

Design and Implement of a
Blockchain-Based System to Prevent
Tampering in Online Public Opinion
Poll

지도교수 김 수 현

이 논문을 공학사학위 논문으로 제출함

2025년 06월 13일

순천향대학교 SW융합대학
컴퓨터소프트웨어공학과

백승훈

백 승 훈의 공학사학위논문을 인준함

2025년 06월 13일

심 사 위 원 김 수 현 인

순천향대학교 SW융합대학

컴퓨터소프트웨어공학과

차 례

그림 차례	ii
표 차례	iv
초 록	1
제 1 장 서 론	2
1.1 연구 배경	2
1.2 연구 목표	4
제 2 장 관련 연구	6
2.1 온라인 여론조사	6
2.2 기존 온라인 여론조사 분석	7
2.2.1 서울시 여론조사	7
2.2.2 청주시선	8
2.2.3 기존 블록체인 여론조사	9
2.3 개발 도구	10
2.3.1 프론트엔드	11
2.3.2 백엔드 및 블록체인 연동	12
2.3.3 스마트 컨트랙트 개발	14
2.3.4 IDE 및 검증 도구	16
제 3 장 시스템 요구사항	18
제 4 장 제안방식 설계 및 구현	19
4.1 아이템 시나리오	19
4.2 제안방식 구현	25
4.3 제안방식 분석	36
제 5 장 결론 및 향후 계획	39
[참고문헌]	41
[부 록]	42

그림 차례

[그림 1-1] 여론조사의 필요성 및 인식 조사	2
[그림 1-2] 여론조사의 신뢰도 조사	3
[그림 1-3] 여론조사의 조작 의심 조사	3
[그림 1-4] 연구 목표	4
[그림 2-1] 기존 온라인 여론조사 동작 과정	6
[그림 2-2] 서울시 여론조사 G밸리 인지도 조사 결과	8
[그림 2-3] 청주시선의 무심천, 미호강 활용 여론조사 결과	8
[그림 2-4] 기존 블록체인 여론조사 시나리오	9
[그림 2-5] 프론트엔드 개발 도구	1
[그림 2-6] 백엔드 및 블록체인 연동 도구	2
[그림 2-7] 스마트 컨트랙트 개발 도구	4
[그림 2-8] IDE 및 검증 도구	6
[그림 4-1] 전체 시나리오	9
[그림 4-2] 스마트 컨트랙트 작성 및 블록체인 배포	2
[그림 4-3] DApp 웹사이트 개발 및 MetaMask 인증 연동	2
[그림 4-4] 설문 기획자에 의한 설문 작성 및 등록	2
[그림 4-5] 사용자에게 의한 설문 참여 및 응답 제출	2
[그림 4-6] 블록체인에 응답 데이터 저장 및 무결성 확보	3
[그림 4-7] Etherscan을 통한 검증	42
[그림 4-8] 설문 종료 후 결과 처리 및 참여 검증	4
[그림 4-9] truffle-config.js 일부 코드	52
[그림 4-10] SurveyFactory.sol 일부 코드	62
[그림 4-11] MetaMask 연동 및 지갑 연결 로직 코드	72
[그림 4-12] 블록체인 기반 온라인 여론조사 시스템 메인페이지	82
[그림 4-13] 생성 페이지	92
[그림 4-14] 문항 생성 세부 화면	92
[그림 4-15] 트랜잭션 요청 화면	93
[그림 4-16] 생성 완료 안내	93

[그림 4-17] 응답 가능 설문 목록 페이지	B
[그림 4-18] 응답 페이지	2
[그림 4-19] 설문 응답 완료 안내	3
[그림 4-20] 결과 확인 페이지	3
[그림 4-21] 결과 페이지	4
[그림 4-22] Etherscan 검증 화면	53
[그림 4-23] MetaMask 지갑 주소	53
[그림 5-1] 기대효과	8

표 차례

<표 4-1> 제안 아이 템 비 교 분 석 표	37
---------------------------------	----

초 록

우리 사회에 있어서 여론조사는 구성원 간의 의사소통을 원활하게 하고, 정책 결정 과정에서 정당성을 확보하며 민주주의를 발전시키고 유지하는 데 필수적인 수단으로 자리 잡아왔다. 그러나 기존의 중앙집중형 구조의 여론조사 시스템은 기관의 신뢰도에 의존하며, 결과의 위·변조 가능성과 불투명한 관리 구조로 인하여 결과 데이터의 무결성이 훼손될 수 있다는 문제점이 존재한다. 이러한 문제는 여론조사 결과에 대한 사회 구성원들의 신뢰 저하 및 참여율 감소로 이어질 수 있으며, 궁극적으로 사회적 합의 과정에서 부정적 영향을 미칠 우려가 있다.

본 논문에서는 블록체인 기술의 핵심 특성인 분산화와 투명성, 불변성을 활용하여 기존 시스템의 문제점을 해결할 수 있는 온라인 여론조사 시스템을 제안한다. 제안 시스템은 이더리움을 기반으로 한 스마트 컨트랙트를 이용하여 여론조사 생성, 응답, 결과 집계 등의 핵심 과정을 온체인으로 처리하여 응답 내역의 위·변조 문제를 방지한다. 특히 모든 설문과 응답 트랜잭션 기록은 블록체인 탐색기를 통하여 누구나 확인할 수 있기에 운영 절차의 투명성을 확보하고, 스마트 컨트랙트를 통한 중복 응답 방지 및 응답자 인증을 지원한다.

제안 시스템의 구현은 스마트 컨트랙트와 React.js 기반의 프론트엔드로 구성하였다. 설문 생성 시 입력된 제목, 설명, 유형, 마감 기간을 컨트랙트를 통해 설문을 배포하고 응답자는 해당 설문 주소로 접근하여 블록체인 네트워크에 응답 데이터를 기록한다. 마감 후 자동 결과 집계를 통해 응답 분포를 시각화하여 사용자에게 제공할 수 있도록 한다.

본 논문은 블록체인 기반 조작 방지 온라인 여론조사 시스템의 설계 및 구현 사례를 제시함으로써 결과 데이터의 신뢰성과 투명성을 현저히 개선하는 것을 목표로 하였다. 또한, 공공 및 민간 기관의 의사결정 과정에서 신뢰할 수 있는 기술적 대안을 제공하고, 향후 학술 연구 및 실제 서비스 적용의 기초 자료로 활용될 수 있다.

제 1 장 서 론

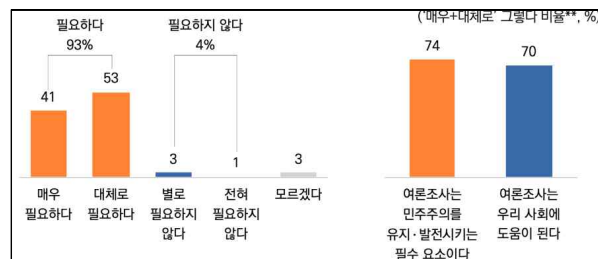
본 장에서는 본 논문에서 제안하고자 하는 온라인 여론조사의 조작 방지를 위한 블록체인 기반 시스템 설계 및 구현의 배경 및 필요성에 관하여 기술한다. 기존 온라인 여론조사의 문제점을 본 아이템을 통해 해결하는 방법을 기술한다.

1.1 연구 배경

여론조사는 특정 사회집단의 구성원을 대상으로 여론의 흐름을 파악하기 위해 수행되는 통계적 사회 조사이자 기법을 의미한다. 이는 [그림 1-1]과 같이 국가 혹은 사회적 사안에 대한 구성원들의 인식이나 행동을 파악하기 위한 목적으로 활용되며, 여론조사의 필요성은 다양한 관점에서 지속적으로 제기되어 왔다[1].

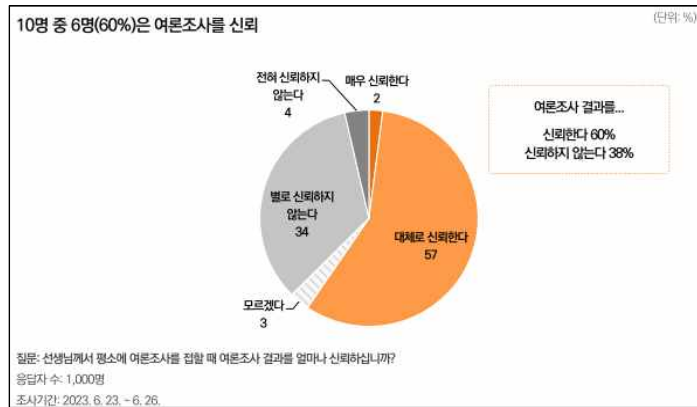
여론조사는 수행 방식에 따라 대면 조사, 전화면접 조사, ARS(Automated Response System) 조사, 온라인 조사, 우편물 조사로 총 다섯 가지 유형으로 분류된다. 대면 조사, 전화 면접조사 등은 시·공간적 제약과 결과 편향 가능성이 존재하기에 본 논문에서는 시·공간적 제약 없이 멀티미디어 활용이 가능한 온라인 여론조사 방식을 채택하여 시스템을 구현하고자 한다.

현재 대부분의 온라인 여론조사 시스템은 응답을 중앙 서버에 저장하고 관리하기 때문에, 중복 응답 및 위·변조 가능성과 같은 보안상의 한계를 내포하고 있으며, 검증 기관의 의존성 문제가 존재한다. 이러한 구조는 통계 결과의 신뢰도를 떨어뜨리고, 여론조사 참여율 저하로 이어질 수 있는 문제를 초래한다.



[그림 1-1] 여론조사의 필요성 및 인식 조사

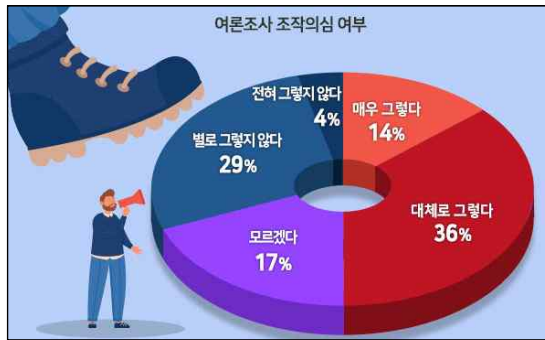
<출처: 한국 리서치 여론 속의 여론, 2023.08>



[그림 1-2] 여론조사의 신뢰도 조사

<출처: 한국리서치 여론 속의 여론, 2023.08>

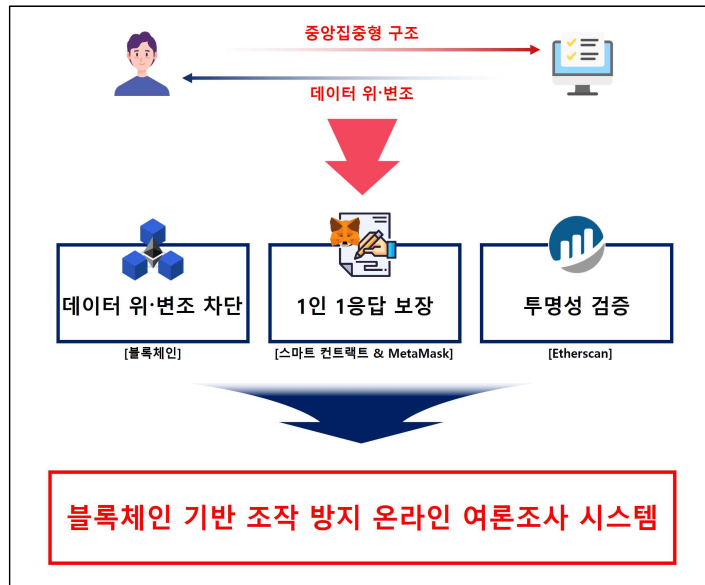
기존 방식의 문제는 중앙 서버의 오류나 외부로부터의 공격이 발생할 경우, 전체 응답 결과의 무결성이 침해될 수 있다. 이는 단순한 기술적 결함을 넘어, [그림 1-2]와 같이 여론조사 결과에 기반한 정책이나 사회적 합의 형성과 같은 민주적 의사결정 과정의 신뢰성을 근본적으로 훼손할 수 있다. 이는 여론조사 결과의 오염 가능성을 높이며, 데이터 손실이나 결과 조작 등의 문제가 발생할 경우 이를 검증하거나 복원하는 것을 어렵게 한다[2]. 따라서 이러한 구조적 한계를 극복하고, [그림 1-3]에서 보여주는 바와 같이 여론조사의 조작 문제를 해결하며, 나아가 공정성과 신뢰성을 제고할 수 있는 기술적 대안 마련이 필수적이다 [3].



[그림 1-3] 여론조사 조작 의심 조사
 <출처: 데이터숨, 2023.08>

1.2 연구 목표

[그림 1-4]와 같이 기존 중앙집중형 온라인 여론조사 시스템은 데이터 위·변조 가능성과 검증 기관 의존으로 객관성과 독립성이 훼손될 수 있다. 이러한 문제를 해결하기 위해 투명성과 무결성을 보장하는 기술이 필수적이다. 이에 본 논문에서는 블록체인 기술의 핵심 특성인 탈중앙화, 투명성, 불변성을 활용하여 신뢰성과 투명성을 확보한 온라인 설문 참여 및 결과 검증 메커니즘을 설계하고 구현하는 것을 목표로 한다. 기존 중앙집중형 여론조사 시스템은 설문 응답이 단일 서버에 의해 관리되면서 위·변조 및 중앙 서버 의존성으로 인한 취약점이 존재한다. 이를 해결하기 위해 본 연구에서는 블록체인의 탈중앙화 특성을 활용하여 스마트 컨트랙트 기반 설문 데이터가 다수의 P2P(Peer to Peer) 노드에서 분산 저장되도록 함으로써 중앙 서버 의존의 취약성을 제거한다. 또한, 불변성 특성으로 한 번 기록된 응답은 변경이 불가능하도록 영구 저장하여 데이터 무결성을 보장하며, 투명성 특성으로 모든 응답 이력을 공개적으로 조회할 수 있게 하여 검증 과정을 간소화한다. 나아가 참여자의 지갑 주소를 고유 식별자로 활용하여 중복 응답을 방지함으로써 여론조사 결과의 정확성과 신뢰도를 향상하고자



[그림 1-4] 연구 목표

한다.

프론트엔드는 React.js를 기반으로 제작하고, Web3.js를 통해 블록체인과 연동하여 사용자 인터페이스와 블록체인 간의 상호작용을 지원한다. 전체 시스템은 이더리움 세폴리아 테스트넷 및 Infura 노드를 기반으로 블록체인 네트워크 위에서 스마트 컨트랙트를 통해 핵심 로직을 처리하고, 데이터의 투명성과 불변성을 보장하는 웹 어플리케이션인 DApp(Decentralized Application)을 구축한다. 시스템의 설문 생성부터 참여, 결과 확인에 이르는 전 과정을 실제 환경에서 테스트하여, 제안하는 시스템의 실용성과 확장 가능성을 검토하고자 한다.

궁극적으로 이러한 연구를 통해 응답 결과에 대한 신뢰성과 투명성이 확보된 블록체인 기반 온라인 설문 시스템의 현실적 구현 가능성을 입증하고, 이를 대학 내 여론조사, 공공기관 의견 수렴 등 다양한 분야에 적용하여 객관적이고 독립적인 여론 수렴 과정에 기여하며, 블록체인 기반 온라인 여론조사 시스템의 사회적 가치에 대한 인식을 고취하는 데 기여하고자 한다.

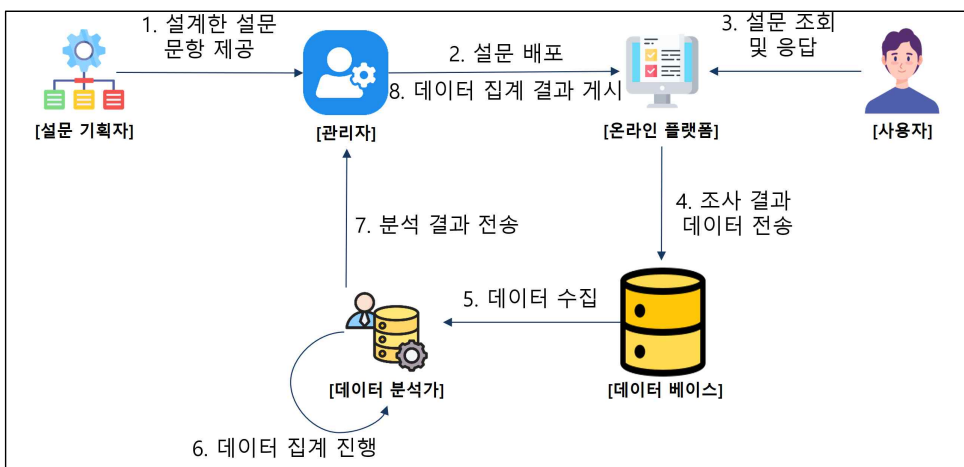
제 2 장 관련 연구

본 장에서는 온라인 여론조사의 조작 방지를 위한 블록체인 기반 시스템을 구현하는 데 필요한 관련 연구들에 대해서 알아본다. 기존 온라인 여론조사에 발생하는 문제점을 언급하고, 이를 해결하기 위해 제안 아이템에 적용할 기술에 대해 설명한다.

2.1 온라인 여론조사

여론조사는 사회 구성원의 의견, 태도, 가치관 등을 설문 응답을 통해 수집·분석하여 공공정책 수립, 시장 분석, 사회 현상 연구 등에 활용되는 주요 조사 방법이다. 특히 현대 민주주의 사회에서 대중의 의사를 반영하고 정책의 정당성과 수용성을 확보하는 데 핵심적인 역할을 한다.

최근 기술 발전과 디지털 기기의 보급으로 온라인 여론조사가 활발해졌다. 온라인 방식은 시·공간적 제약이 없고, 텍스트, 이미지, 영상 등 다양한 멀티미디어 활용이 가능해 응답자 확보가 용이하며, 물리적 자원 소모를 줄여 효율적인 조사 방식으로 주목받고 있다[4].



[그림 2-1] 기존 온라인 여론조사 동작 과정

현재의 온라인 여론조사 시스템은 [그림 2-1]과 같이 동작한다. 설문 기획자가 조사 목적에 맞추어 문항을 설계하면, 관리자는 이를 온라인 플랫폼에 등록하여 설문을 배포한다. 배포된 설문은 사용자가 플랫폼에 접속하여 조회하고 답변을 제출하면, 응답 데이터가 플랫폼의 서버를 통해 데이터베이스에 저장된다. 저장된 응답 데이터는 데이터 분석가에 의해 추출 및 정제된 후 통계적 집계 과정을 거친 후 분석 결과로 가공되며, 해당 결과는 관리자의 검토를 거쳐 플랫폼에 올라가게 된다. 게시된 결과는 설문 기획자, 관리자, 사용자 모두가 언제든지 조회할 수 있도록 제공된다.

2.2 기존 온라인 여론조사 분석

본 절에서는 본 논문에서 제안하는 시스템을 개발하기 위해 기존의 온라인 여론조사 시스템을 분석하고, 이에 따라 발생하는 다양한 문제점을 알아본다. 또한, 기존의 문제점을 보완하기 위해 본 논문에서 제안하는 시스템과 유사한 시스템을 분석한다. 이러한 기존 온라인 여론조사의 문제점을 분석하고, 여론조사 결과에 대한 신뢰성 확보 및 사회적 의사결정 과정에 대한 대중 인식 제고를 위한 방안을 탐구한다.

2.2.1 서울시 여론조사

‘서울시 여론조사’는 온라인으로 사용자들의 의견을 수집하여 분석하고, 서울시 복지 정책 및 사회 문제 결정에 활용되는 결과를 제공한다. 또한, [그림 2-2]와 같이, 수집된 데이터를 바탕으로 시민들의 의견을 정책에 반영하고 통계 분석 결과를 시민들에게 제공한다[5].

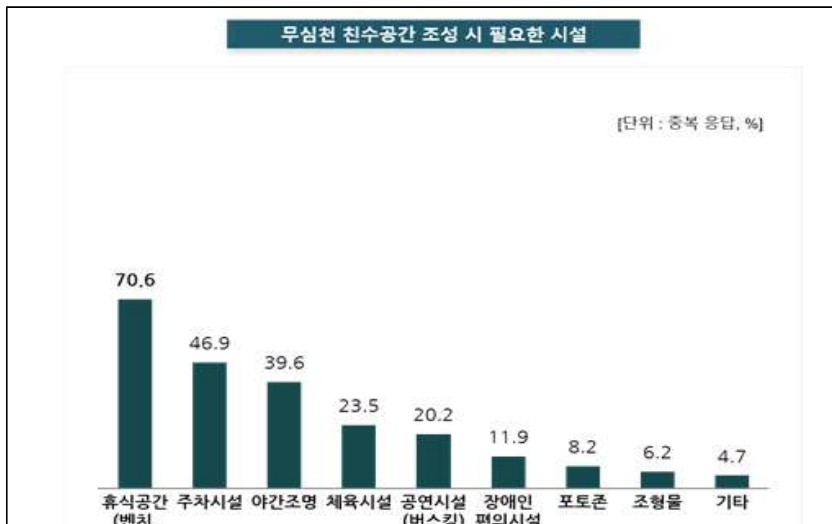
‘서울시 여론조사’는 구분을 다양하게 나누고 있지만, 중앙 서버에서 결과 분석 및 통계가 수행되기 때문에, 결과가 조작되었다 하더라도 본인의 선택이 제대로 반영되었는지 확인할 수 없다는 결함이 존재한다.

구분		사례수	G밸리	서울디지털 국가산업단 지	구로디지털 단지	가산디지털 단지	구로공단	아는 명칭 없음
전체		2,566	54.1	32.3	89.4	90.5	77.2	0.4
성별	남	1,469	53.6	36.5	89.0	90.6	77.9	0.3
	여	1,096	54.7	26.6	89.9	90.4	76.2	0.4
연령	10대	12	56.3	25.0	66.7	100.0	58.3	0.0
	20대	162	63.0	19.8	91.4	91.4	64.2	1.2
	30대	533	56.1	19.1	92.5	91.0	70.2	0.4
	40대	769	55.0	28.1	87.4	90.6	76.5	0.4
	50대	546	53.3	39.4	89.7	93.2	84.2	0.4
	60대 이상	544	49.1	47.8	88.8	86.8	82.4	0.0

[그림 2-2] ‘서울시 여론조사’의 G밸리 인지도 조사 결과
 <출처: 서울시 여론조사 G밸리 인지도 조사, 2021.11>

2.2.2 청주시선

‘청주시선’은 [그림 2-3]과 같이 여론조사 결과를 통계 자료와 시각적 리포트 형태로 제공한다[6]. 또한, 여론조사를 통한 반영 여부를 ‘반영/일부 반영/참고’의 항목으로 나누어 표시해 직접적으로 어떻게 적용이 되었는지 확인할 수 있다.

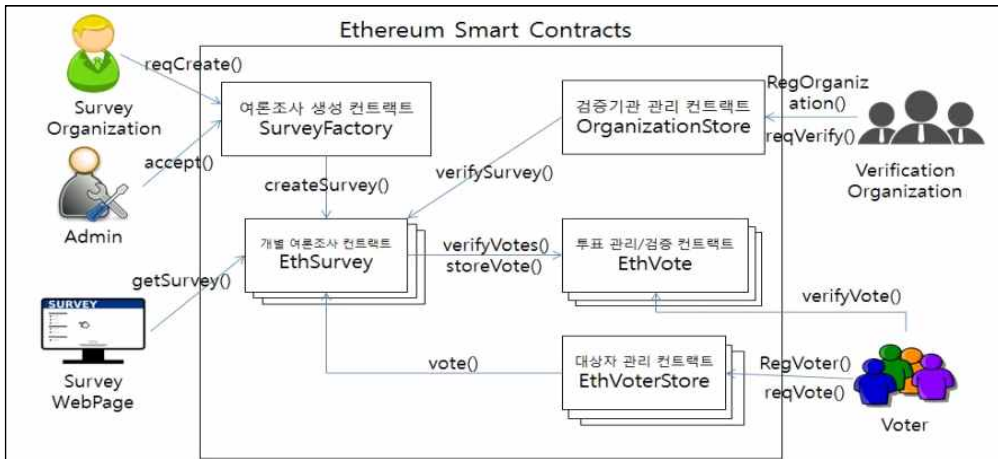


[그림 2-3] ‘청주시선’의 무심천, 미호강 활용 여론조사 결과
 <출처: 청주시선 무심천, 미호강 활용을 위한 청주시민 의견조사, 2022.07>

또한, 결과에 대한 통계를 그래프, 카드 뉴스 등의 형태로 집계하여 제공한다. ‘청주시선’은 여론조사의 내용에 따라 결과 산출물이 달라지는데, 사이트 자체에서 메뉴 항목이 많다 보니, 원하는 결과를 보기 위해서는 일일이 찾아야 한다는 번거로움이 존재한다. 보고서 형식으로 나온 여론조사 결과는 사람들의 전체적인 의견을 확인할 수 없으며, 글과 사진만 첨부되어 있기에, 본인의 응답이 결과에 반영되었는지에 대한 투명성이 반영되지 않는다. 또한, ‘서울시 여론조사’와 마찬가지로, 본인의 항목 선택에 대한 값이 제대로 반영되었는지 확인하기 어렵다. 그로 인해 전체적인 데이터 무결성과 투명성이 충분히 확보되지 않는다.

2.2.3 기존 블록체인 기반의 여론조사

2018년 진제환 외 3인이 발표한 「견고한 검증을 제공하는 이더리움 블록체인 기반의 여론조사 어플리케이션」에서는, 2.2.1절과 2.2.2절에서 언급한 기존 여론조사의 조작 가능성과 신뢰성 문제를 해결하기 위한 방안으로 이더리움 스마트 컨트랙트를 기반으로 하는 분산 어플리케이션을 제안하였다. 이 시스템은 [그림 2-4]와 같이 여론조사 생성부터 응답, 검증까지의 과정을 블록체인에 기록함으로써 결과의 위·변조를 방지하고 응답 무결성을 확보할 수 있도록 설계되었다[7].



[그림 2-4] 기존 블록체인 여론조사 시나리오

특히, 해당 논문에서는 응답자가 본인의 여론조사 데이터를 개인 키로 암호화하여 제출하고, 이를 블록체인에 저장하므로, 검증 기관이나 당사자가 복호화 및 서명 검증을 통해 응답이 정확히 반영되었는지 확인할 수 있도록 구현하였다. 이러한 방식은 2.2.1절과 2.2.2절에서 언급한 ‘서울시 여론조사’ 및 ‘청주시선’에서 나타난 ‘본인의 응답이 결과에 반영되었는지 확인할 수 없는 문제’, ‘결과 집계계의 투명성 부족’ 등을 기술적으로 보완할 수 있다.

그러나 해당 시스템 역시 현실적인 한계를 내포하고 있다. 논문에서 제안한 시스템은 프라이빗 블록체인 환경에서 동작하며, 실제 대규모의 공공 여론조사에 적용하기 위해서는 기술적 인프라, 사용자의 암호화 키 관리, 검증 기관의 신뢰성 확보 등에서 여전히 해결 과제가 존재한다. 또한, 이메일 인증 기반의 유권자 등록 방식은 인증 단계에서의 위·변조 가능성을 완전히 배제하기 어려우며, 일반 시민이 직접 설문 내역을 검증하기 위한 절차도 기술적 이해도를 요구한다는 점에서 접근성이 떨어질 수 있다.

결론적으로, 해당 연구는 블록체인 기반 여론조사의 기술적 가능성과 장점을 실증적으로 보여주었지만, 지금까지는 일부 한계와 현실 적용에 대한 고려가 필요하다는 점에서 기존 시스템의 보완책이자, 동시에 새로운 과제를 제시한 사례라고 볼 수 있다.

2.3 개발 도구

본 절에서는 앞선 기존 연구의 문제들을 해결하는 블록체인 기반 온라인 여론조사 시스템을 설계하고 구현하기 위해 사용된 개발 도구에 대해 설명한다. 프론트엔드와 백엔드, 스마트 컨트랙트 개발 환경, 블록체인 네트워크 구성 요소, 그리고 통합 개발 환경으로 구분하여 기술한다.

2.3.1 프론트엔드

프론트엔드는 사용자와 직접 상호작용하는 웹 인터페이스로, 여론조사 생성, 참여, 결과 확인 등 주요 기능을 시각적으로 제공한다. 본 논문에서는 React.js 기반으로 구현하며, 사용자 편의성과 직관적인 설문 흐름을 고려하여 구성한다. 이는 [그림 2-5]의 개발 도구들을 활용한다.

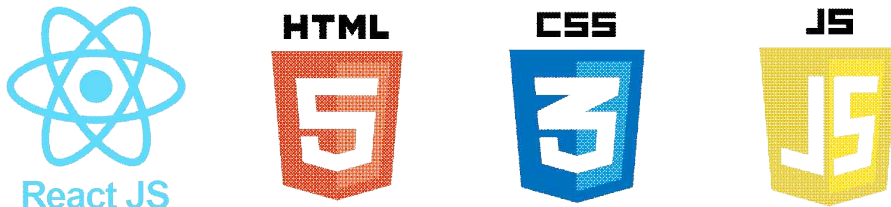
1) React.js

React.js는 JavaScript 기반의 UI(User Interface) 라이브러리로, 컴포넌트 구조를 통해 화면을 효율적으로 구성하고 상태 변화에 따라 자동 갱신되는 특징이 있다[8]. 본 시스템에서는 설문 생성, 참여, 결과 확인 등 다양한 화면을 구성하는 데 사용한다.

2) HTML

HTML(HyperText Markup Language)은 웹 페이지의 구조를 정의하는 마크업 언어로, 텍스트, 버튼, 입력창 등의 요소를 구분한다[9]. 본 시스템에서는 설

문 제목, 설명, 질문 항목 등 기본 틀을 설계하는 데 활용한다.



[그림 2-5] 프론트엔드 개발 도구

3) CSS

CSS(Cascading Style Sheets)는 웹 페이지의 시각적 요소를 꾸미는 스타일 언어로, 색상, 배치, 글꼴 등을 설정한다[10]. 본 시스템에서는 설문 화면의 가독성과 디자인을 향상시키기 위해 사용한다.

4) JavaScript

JavaScript는 웹 페이지에 동작을 부여하는 언어로, 입력 처리, 이벤트 제어, MetaMask 연동 등을 구현한다[11]. 본 시스템에서는 설문 응답 처리와 트랜잭션 발행을 담당한다.

2.3.2 백엔드 및 블록체인 연동

백엔드에서는 사용자의 설문 응답을 블록체인에 기록하고, 해당 데이터를 조회하는 기능을 Web3.js와 스마트 컨트랙트를 통해 수행한다. 서버를 따로 두는 대신, 탈중앙화된 블록체인 네트워크를 데이터 저장소로 활용함으로써, 시스템의 투명성과 신뢰성을 확보한다. 이는 [그림 2-6]의 도구들과 같다.



[그림 2-6] 백엔드 및 블록체인 연동 도구

1) Web3.js

Web3.js는 이더리움 기반 DApp을 개발하기 위한 대표적인 JavaScript 라이브러리로, 스마트 컨트랙트와의 상호작용이 가능하도록 다양한 API(Application Programming Interface)를 제공한다[12]. 이 라이브러리를 통해 JavaScript 코드 내에서 이더리움 네트워크에 연결하고, 컨트랙트 메서드를 호출하거나 트랜잭션을 전송할 수 있다. Web3.js는 Web3 생태계의 핵심 요소로 분산 네트워크와 상호작용하기 위한 주요 도구이다[13]. 본 시스템에서는 여론조사에 대한 참여, 응답 기록, 결과 조회 등 주요 기능을 스마트 컨트랙트와 연결하여 처리하기 위해 Web3.js를 활용한다. 예를 들어, 사용자가 특정 질문에 응답을 제출하면 Web3.js를 통해 스마트 컨트랙트의 관련 함수가 호출되며, 해당 트랜잭션은 블록체인에 기록한다. 이를 통해 사용자의 여론조사 응답의 무결성과 투명성을 확보할 수 있다. 또한, Web3.js는 MetaMask 지갑과 연동되어 사용자의 계정 정보 확인 및 서명 요청 기능을 수행한다.

2) MetaMask

MetaMask는 사용자들이 브라우저 환경에서 이더리움 지갑을 생성하고 관리할 수 있도록 도와주는 확장 프로그램으로, DApp과의 연결을 위한 핵심 도구 중 하나

나이다. 사용자는 MetaMask를 통해 개인 지갑을 생성하고, 공개키 및 개인 키 기반의 블록체인 계정을 관리할 수 있다[14]. 제안하고자 하는 본 시스템은 여론조사에 참여하는 사용자의 신원을 MetaMask의 계정 주소를 기준으로 대체 인증함으로써, 개인정보를 수집하지 않고도 사용자의 식별 및 권한 제어가 가능하도록 구현한다. 또한, MetaMask는 트랜잭션 서명 기능을 제공함으로써, 사용자가 설문 응답을 제출할 때 실제로 해당 내용을 블록체인에 기록하도록 승인할 수 있도록 한다. 아울러, 스마트 컨트랙트 내부에서 해당 주소로 중복 응답 여부를 확인하여 동일 계정의 다중 참여를 방지하는 기능도 함께 구현할 수 있다.

3) Infura

Infura는 원격 이더리움 노드를 통해 네트워크에 접근할 수 있도록 해주는 서비스이다. RPC(Remote Procedure Call) URL(Uniform Resource Locator)과 API key를 제공하기 때문에, 네트워크에 직접 블록을 동기화하지 않아도 접근할 수 있다[15]. 이더리움 네트워크에 직접 연결하려면 풀 노드를 구성하고 유지하는 비용과 기술적 부담이 크기 때문에, Infura는 개발자들이 손쉽게 블록체인 어플리케이션을 구현할 수 있도록 하는 매우 유용한 도구로 자리 잡고 있다. 본 시스템에서는 스마트 컨트랙트를 이더리움 세폴리아 테스트넷에 배포하고, 사용자가 설문을 생성하거나 응답할 때 해당 컨트랙트와의 연결을 위해 Infura에서 제공하는 RPC URL을 활용한다. Infura를 통해 DApp은 안정적으로 블록체인 네트워크와 통신할 수 있으며, 다양한 사용자가 동시에 접근하더라도 노드 연결 이슈 없이 트랜잭션 요청을 처리할 수 있도록 하여 서비스의 안정성과 확장성을 높인다.

2.3.3 스마트 컨트랙트 개발

여론조사 생성을 위한 로직은 Solidity 언어로 작성된 스마트 컨트랙트를 통해 구현되며, 설문의 무결성과 중복 방지 등을 자동으로 처리하도록 설계한다. 이는 [그림 2-7]의 도구들과 같다.



SOLIDITY



[그림 2-7] 스마트 컨트랙트 개발 도구

1) Solidity

Solidity는 이더리움 기반의 스마트 컨트랙트를 작성하기 위한 고급 프로그래밍 언어로, JavaScript, Python, C++ 등의 언어에서 영향을 받아 설계되었다 [16]. 정적 타입 언어로서 계약 기반의 프로그래밍을 지향하며, 스마트 컨트랙트의 보안성과 신뢰성을 높이는 데 적합한 구조를 갖추고 있다. 이 언어는 EVM(Ethereum Virtual Machine)에서 실행되기 때문에, 블록체인 상의 분산 환경에서 상태를 변경하거나 트랜잭션을 발생시키는 로직을 구현할 수 있다[17].

본 시스템에서는 Survey와 SurveyFactory라는 두 개의 핵심 스마트 컨트랙트를 Solidity로 구현한다. SurveyFactory은 설문을 생성하는 기능을 담당하며, 생성된 설문은 독립된 Survey 컨트랙트로 배포한다. 각 Survey 컨트랙트는 응답 등록, 중복 응답 방지, 마감일 처리, 결과 조회 등의 로직을 포함하고 있으며, 이를 통해 관리자가 임의로 결과를 수정할 수 없는 투명한 여론조사 환경을 제공한다. Solidity를 활용함으로써 설문 참여 이력의 무결성과 조작 불가능성을 보장할 수 있으며, 모든 기능이 분산 네트워크 위에서 자동으로 실행한다.

2) Truffle

Truffle은 이더리움 기반의 스마트 컨트랙트를 개발하고 테스트하며 배포하는데 사용되는 대표적인 개발 프레임워크이다[18]. Truffle은 Solidity 코드를 컴

파일하고, 네트워크에 배포하며, 배포 후 테스트를 자동화할 수 있는 환경을 제공한다. 또한, 컨트랙트 배포 이력을 관리할 수 있는 마이그레이션 기능과 테스트를 위한 가상 블록체인 환경도 함께 제공하여 개발자의 생산성을 극대화한다.

본 시스템에서는 Truffle을 통해 작성한 스마트 컨트랙트를 세폴리아 테스트넷에 안전하게 배포하고 관리한다. 먼저 `truffle-config.js`를 설정하여 Infura를 통한 네트워크 연결을 구성한 뒤, 명령어를 통해 SurveyFactory 및 Survey 컨트랙트를 배포한다. 이 과정에서 Truffle의 migrations 디렉터리에 작성한 배포 스크립트(`2_deploy_smart_contract.js`)가 활용되며, 이를 통하여 반복적인 배포 작업이 자동화되도록 하고 체계적으로 관리한다. 또한, Truffle은 개발 중 테스트 코드 실행 및 디버깅을 지원하여, 스마트 컨트랙트의 안정성과 정확성을 사전에 검증할 수 있도록 돕는다[19].

2.3.4 IDE 및 검증 도구

스마트 컨트랙트와 프론트엔드, 백엔드 개발을 효율적으로 관리하기 위해 사용 편의성과 확장성이 뛰어난 Visual Studio Code를 활용하며, 배포된 컨트랙트의 상태 확인 및 트랜잭션 모니터링을 위해 Etherscan을 병행하여 사용한다. 이 도구들은 [그림 2-8]과 같다.

1) Visual Studio Code

Visual Studio Code는 Microsoft에서 개발한 크로스 플랫폼 통합 개발 환경으로, 다양한 확장 기능을 통해 Solidity, JavaScript, React 등 여러 언어와 기술 스택을 지원한다[20]. 본 시스템의 전반적인 개발에 모두 활용하며, 프로젝트 구조를 통합적으로 관리할 수 있는 장점이 있다.



[그림 2-8] IDE 및 검증 도구

2) Etherscan

Etherscan은 이더리움 블록체인 상에서 모든 트랜잭션과 스마트 컨트랙트 등의 정보를 실시간으로 조회할 수 있는 블록 탐색기이다[21]. 배포된 스마트 컨트랙트의 ABI(Application Binary Interface) 확인, 소스 코드 검증 등의 기능을 지원하며, 함수의 호출 내역이나 이벤트 로그를 쉽게 확인할 수 있다. 특히 Etherscan은 네트워크 상의 모든 기록을 검증할 수 있는 형태로 제공함으로써 온체인 데이터의 무결성과 투명성을 보장한다는 장점이 있다.

제 3 장 시스템 요구사항

본 장에서는 여론조사 응답 결과의 위·변조 가능성의 문제를 해결하는 조작 방지 블록체인 기반 온라인 여론조사 DApp의 시스템 요구사항에 대해 기술한다. 해당 시스템은 응답 결과의 무결성과 투명성 보장, 중복 참여 방지, 응답 확인 기능 등을 제공함으로써 기존 중앙집중형 온라인 여론조사의 문제점을 해결하고자 한다. 주요 요구사항은 아래와 같이 세 가지 측면에서 정리할 수 있다.

- **블록체인 기반의 데이터 무결성 및 조작 방지**

본 시스템은 사용자의 여론조사 응답을 스마트 컨트랙트를 통해 블록체인에 기록함으로써, 관리자에 의한 데이터 조작이나 삭제를 원천적으로 차단한다. 사용자는 MetaMask 지갑을 통해 여론조사에 응답하며, 해당 응답 결과는 Etherscan을 통해 검증 가능할 수 있도록 한다. 이를 통해 중앙 관리자에 의한 조작 우려를 제거하고, 무결성을 확보한다.

- **중복 응답 방지 및 사용자 검증**

스마트 컨트랙트는 동일 계정의 중복 참여를 방지하는 로직을 내장하고 있으며, 사용자는 설문 참여 시 MetaMask 지갑 주소를 통해 인증된다. 하나의 지갑 주소로 한 번만 응답할 수 있도록 제한하여 설문의 신뢰성과 공정성을 높이며, 각 사용자의 응답은 해당 주소에 연결되어 블록체인 상에 안전하게 기록된다.

- **응답 결과 확인 및 사용자 피드백 기능**

기존 블록체인 여론조사의 결과 반영 여부를 해결하기 위해 사용자는 본인의 응답이 반영되었는지 DApp에서 제공하는 결과를 통해 확인할 수 있다. 결과 페이지에서 사용자는 본인의 응답 여부를 확인할 수 있고, 그래프 및 응답 강조를 통해 명확히 확인할 수 있다. 이를 통해 사용자는 자신의 의견 반영 여부를 확인할 수 있고, 이를 통해 여론조사 신뢰도를 제고할 수 있다.

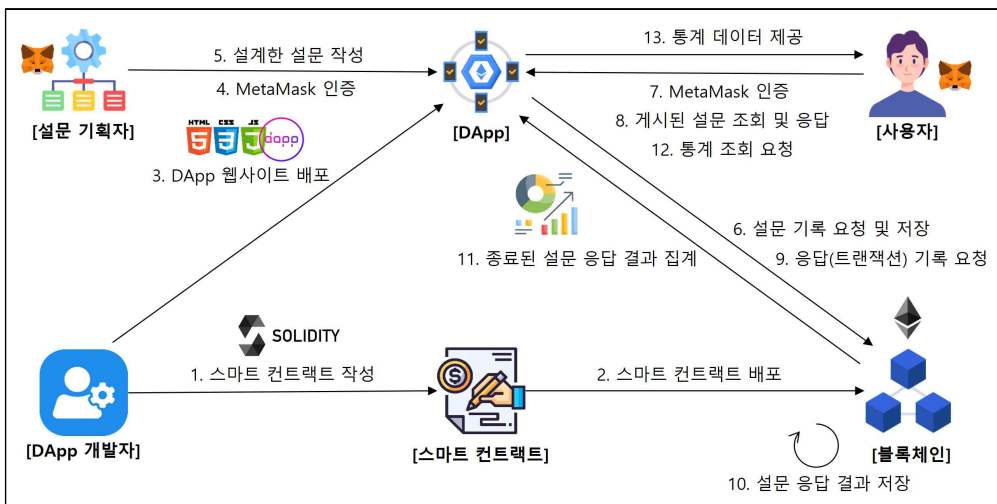
제 4 장 제안방식 설계 및 구현

본 장에서는 2장에서 언급했던 기존 온라인 여론조사의 문제를 해결하기 위한 블록체인 기반 온라인 여론조사 DApp 시스템의 제안방식 설계 및 구현에 대해 기술한다. 본 시스템의 구현을 위해 구성된 전체 시나리오를 기술하고, 이후 각 단계에 대한 세부 구현 내용을 바탕으로 분석을 진행한다.

4.1 아이템 시나리오

본 절에서는 제안하는 블록체인 기반 여론조사 DApp의 전체 시나리오와 세부 시나리오를 다룬다. 전체 시나리오에서는 설문 기획자, 사용자, DApp 개발자의 흐름에 따라 전체적인 동작 과정을 설명하고, 세부 시나리오에서는 각 참여 주체의 세부 절차를 구체적으로 설명한다.

1) 전체 시나리오



[그림 4-1] 전체 시나리오

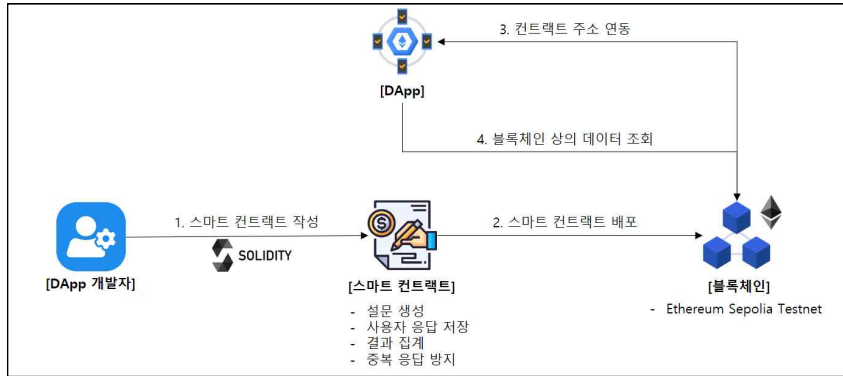
제안하는 블록체인 기반 온라인 여론조사 시스템의 전체적인 시나리오는 [그림 4-1]과 같다. 본 시스템의 주요 객체로는 설문 기획자, 사용자, DApp 개발자가 참여하고 상호작용한다. 먼저 DApp 개발자는 Solidity 언어를 기반으로 설문 생성을 위한 스마트 컨트랙트와 개별 설문을 관리하는 스마트 컨트랙트를 작성한 후 이더리움 블록체인에 배포한다.

설문 기획자는 배포된 DApp을 통해 여론조사를 기획하고 등록하며, 여론조사 기간과 질문 항목 등을 설정한다. 사용자는 DApp에 접속하여 MetaMask를 통해 본인 인증을 수행한 뒤, 게시된 설문 목록을 조회하고 원하는 질문에 응답한다. 응답을 마친 후, 사용자의 입력은 트랜잭션 형태로 스마트 컨트랙트에 전달되며, 합의 과정을 거쳐 유효성 검증이 된 뒤, 이 내용은 블록체인에 저장된다. 설문 종료되면, DApp은 블록체인에 저장된 데이터를 기반으로 결과를 집계하고, 사용자의 요청에 따라 통계 데이터를 제공한다. 이러한 구조를 통해 설문 기획자와 사용자 모두 응답의 무결성과 투명성을 직접 검증할 수 있다.

2) 세부 시나리오

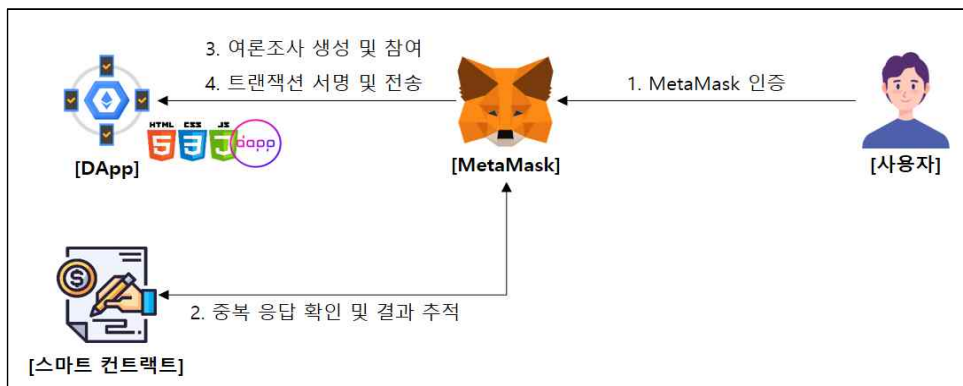
앞서 제시한 전체 시나리오를 기반으로, 제안하는 시스템의 실제 구현 과정을 단계별로 세분화하여 기술한다. 설문 시스템은 설문 기획자, 사용자, 그리고 DApp 개발자라는 세 주체가 각각의 역할을 수행하며, 스마트 컨트랙트와 블록체인 네트워크를 통해 설문 생성, 참여, 결과 집계 및 검증 절차가 진행된다. 각 단계는 스마트 컨트랙트와 DApp 간의 데이터 흐름을 기반으로 구성되며, 실제 사용자 경험을 고려한 설계와 보안성을 강화하는 인증 절차가 포함된다.

가장 먼저, [그림 4-2]와 같이 DApp 개발자는 이더리움 기반의 스마트 컨트랙트를 작성한다. 해당 스마트 컨트랙트는 Solidity 언어를 기반으로 작성되며, 설문 생성, 응답 저장, 결과 집계 등을 수행할 수 있는 기능을 포함한다. 스마트 컨트랙트는 이더리움 세폴리아 테스트넷에 배포되며, 이때 생성된 컨트랙트 주소는 DApp 프론트엔드와의 연동을 위한 고유 식별자로 활용된다. 이를 통해 DApp은 블록체인 상의 데이터를 직접 조회를 수행할 수 있는 구조를 갖추게 된다.

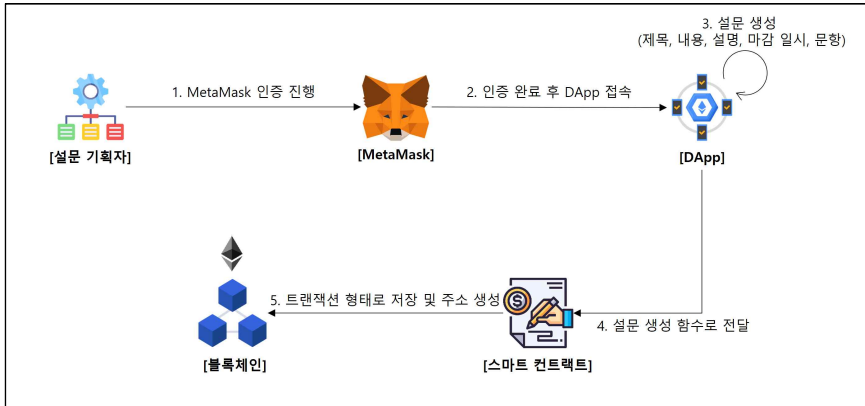


[그림 4-2] 스마트 컨트랙트 작성 및 블록체인 배포

스마트 컨트랙트가 배포된 후, [그림 4-3]처럼 사용자들이 접근할 수 있는 웹 기반의 DApp 인터페이스가 구성된다. 프론트엔드는 HTML, CSS, JavaScript와 Web3.js 라이브러리를 사용하여 개발되며, 사용자와 설문 기획자가 설문을 생성하거나 참여할 수 있도록 UI를 구현하였다. 특히, 사용자의 인증 절차는 중앙화된 로그인 시스템이 아닌 MetaMask 지갑을 기반으로 진행되며, 이를 통해 사용자는 별도의 DApp 회원가입 절차 없이 지갑 주소로 고유하게 식별된다. 이 지갑 주소는 스마트 컨트랙트 상에서 중복 응답 확인 및 결과 추적의 기준이 된다.

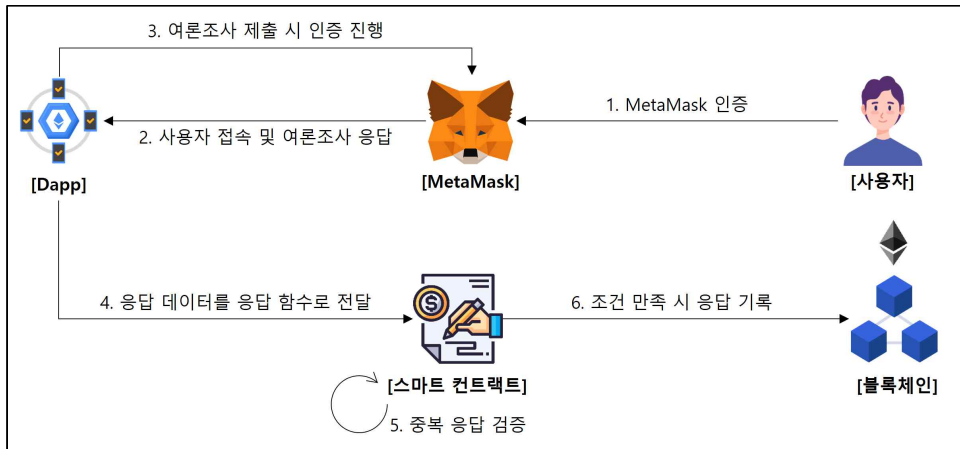


[그림 4-3] DApp 웹사이트 개발 및 MetaMask 인증 연동



[그림 4-4] 설문 기획자에 의한 설문 작성 및 등록

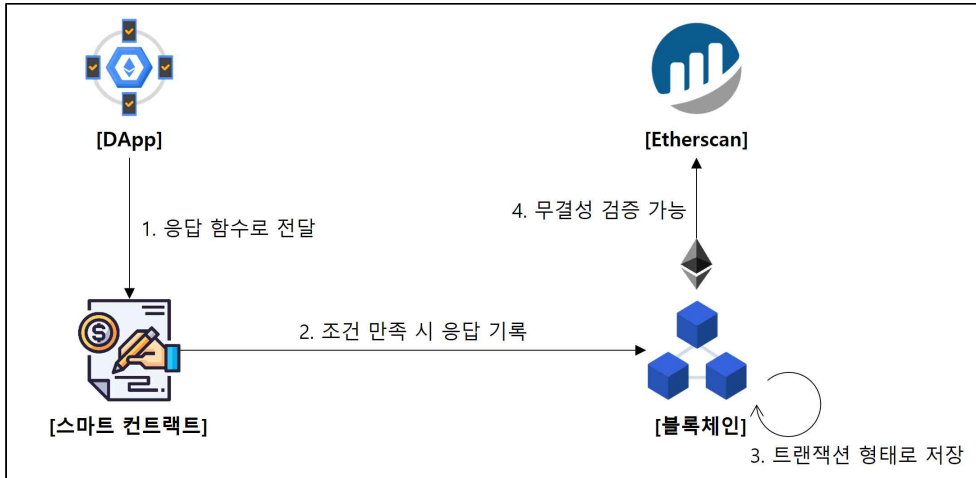
설문 기획자는 MetaMask 인증 절차를 완료한 후, 설문 생성 페이지에서 설문 제목, 설문 설명, 마감일시, 질문 내용, 보기 항목 등을 입력할 수 있다. 이러한 입력값은 [그림 4-4]와 같이 DApp을 통해 스마트 컨트랙트의 생성 함수로 전달되며, 이후 블록체인 상에 트랜잭션 형태로 기록된다. 해당 트랜잭션이 성공적으로 블록에 포함되면, 설문 고유 주소(컨트랙트 주소)가 생성되며, 이는 설문 참여 및 결과 조회 시 참조되는 식별자 역할을 한다. 설문 정보는 블록체인에 영구 저장되므로 시스템 관리자 또는 외부 공격자가 내용을 수정하거나 삭제할 수 없다.



[그림 4-5] 사용자에게 의한 설문 참여 및 응답 제출

[그림 4-5]와 같이 일반 사용자는 DApp에 접속하여 게시된 설문 목록을 확인하고, 원하는 설문을 선택하여 응답할 수 있다. 이때 사용자는 MetaMask를 통해 인증을 수행하며, 해당 사용자의 지갑 주소는 설문 참여자의 고유 식별자로 자동 등록된다. 사용자가 객관식 항목을 선택하거나 주관식 응답을 입력한 후 응답 제출을 클릭하면, DApp은 응답 데이터를 스마트 컨트랙트의 응답 함수로 전달한다. 이 함수는 응답자의 중복 응답 여부를 검증하고, 조건을 만족하는 경우에만 해당 응답을 블록체인에 기록한다. 응답은 트랜잭션 단위로 저장되며, 이후 외부에서 변경할 수 없는 상태가 된다.

[그림 4-6]과 같이 스마트 컨트랙트를 통해 제출된 사용자 응답은 이더리움 블록체인에 트랜잭션 형태로 저장된다. 이때 저장된 응답은 분산 네트워크의 합의 절차를 거쳐 블록에 기록되므로, 중앙 서버 방식과 달리 특정 주체가 데이터에 임의 접근하거나 조작하는 것이 불가능하다. 이를 통해 설문 응답의 무결성과 투명성이 확보되며, 설문 결과에 대한 신뢰도가 높아진다. 특히, 모든 참여자의 응답 내역은 [그림 4-7]과 같은 Etherscan과 같은 블록 탐색기를 통해 공개적으로 검증이 가능하여, 시스템 운영자도 응답을 조작할 수 없는 구조이다.

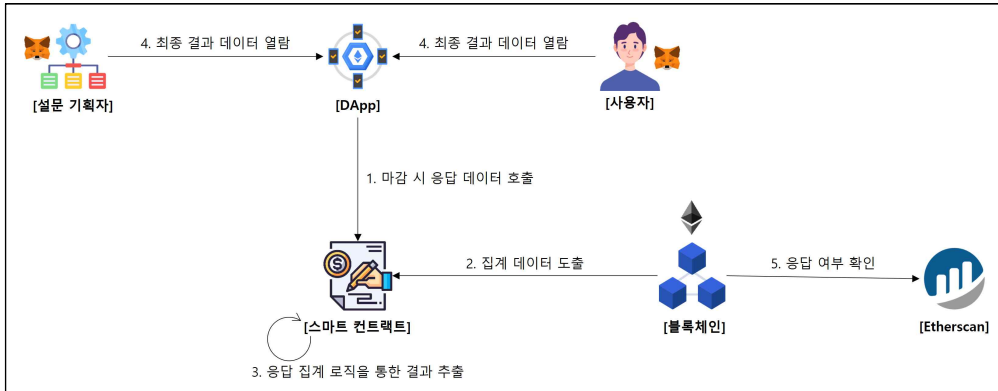


[그림 4-6] 블록체인에 응답 데이터 저장 및 무결성 확보

Transaction Hash:	0xb4f599014c64dca3c1a58e385a4d0001b8e6b4b5227723b84defeb9e669c8b1a
Status:	Success
Block:	8083280 246073 Block Confirmations
Timestamp:	35 days ago (Apr-09-2025 09:55:36 AM UTC)
From:	0x80339C66Ee87F22DbAB41b0d8465bF458c126d01
To:	0xD6c24227A2980a9670EaE965204AFd27e22ca76C [0xbdd92dc199f497d4f6d7229b43817797d442d9e4 Created]
Value:	0 ETH
Transaction Fee:	0.004533893616042524 ETH
Gas Price:	2.526203494 Gwei (0.000000002526203494 ETH)

[그림 4-7] Etherscan을 통한 검증

설문 응답 마감 시간이 되면, [그림 4-8]과 같이 DApp은 해당 설문에 대한 응답 데이터를 스마트 컨트랙트를 통해 호출한다. 컨트랙트 내부에서는 각 보기 항목에 대한 응답 수를 집계하는 로직이 포함되어 있으며, 이를 바탕으로 결과 데이터를 계산한다. DApp은 계산된 결과를 실시간으로 시각화하여 사용자에게 제공하며, 응답 분포를 쉽게 파악할 수 있도록 구성된다. 이 통계 데이터는 사용자 설득, 정책 수립, 의사결정 등에 유용하게 활용될 수 있다.



[그림 4-8] 설문 종료 후 결과 처리 및 참여 검증

또한, 사용자 및 설문 기획자는 결과 페이지를 통해 집계된 데이터를 열람할 수 있다. 이 결과는 블록체인 상의 트랜잭션 데이터를 기반으로 도출되었기 때문에, 누구나 동일한 방식으로 데이터 집계 과정을 재현할 수 있다. 특히 설문 참여자는 Etherscan과 같은 블록 탐색기를 통해 자신이 제출한 응답이 올바르게 반영되었는지를 확인할 수 있다. 트랜잭션 해시와 지갑 주소를 통해 응답 여부를 추적함으로써, 설문 결과에 대한 신뢰도와 투명성을 높일 수 있다.

4.2 제안방식 구현

본 절에서는 4.1절에서 제안방식을 설계한 내용을 바탕으로 구현을 완료한 시스템의 기능과 작동방식에 대해 알아본다.

블록체인 설계 및 구현은 퍼블릭 테스트넷인 이더리움 세폴리아에 배포함으로써 실제 네트워크 환경에서 MetaMask 연동 및 외부 참여자 테스트를 진행하였다. [그림 4-9]와 같이 truffle-config.js 세폴리아 설정 코드에서는 Infura 엔드포인트와 HDWalletProvider를 연결하도록 설정했으며, 네트워크 ID(Identifier)는 1155111, 가스 제한은 4,465,030으로 지정했다. 구현된 환경에서 truffle migration —network sepolia 명령을 실행하면 컨트랙트가 자동으로 배포되며, 배포된 주소와 트랜잭션은 Etherscan에서 확인할 수 있다.

```

networks: {
  sepolia: {
    provider: () =>
      new HDWalletProvider(
        process.env.PRIVATE_KEY,
        `https://sepolia.infura.io/v3/${process.env.INFURA_PROJECT_ID}`
      ),
    network_id: 11155111,
    gas: 4465030,
    confirmations: 2,
    timeoutBlocks: 200,
    skipDryRun: true,
    pollingInterval: 15000,
  },
},

```

[그림 4-9] truffle-config.js 일부 코드

본 시스템의 스마트 컨트랙트는 SurveyFactory.sol 와 Survey.sol 두 개의 스마트 컨트랙트로 구성되어 있다. SurveyFactory.sol은 [그림 4-10]과 같이 createSurvey() 함수를 통해 새로운 Survey 인스턴스를 생성하고 생성된 주소를 배열에 저장한 뒤 SurveyCreated 이벤트를 발생시킨다. 이후, 생성된 여론조사는 전체 여론조사 목록을 반환하는 getAllSurveys() 함수를 통하여 프론트엔드가 배포된 모든 여론조사를 조회할 수 있다.

[그림4-11]과 같이 Survey.sol은 질문을 담는 구조체와 중복 응답을 차단하는 매핑, 응답자의 선택지를 저장하는 매핑을 정의하고, vote 함수는 마감 시간 이전에만 호출하도록 하고, 이미 조사를 완료한 사용자는 다시 조사에 참여하지 못하도록 검증한 뒤에 응답을 기록한다. 결과 집계는 getResults 함수로 구현하여 배열 형태로 반환하고, 프론트엔드가 이를 시각화하여 보여주도록 구현하였다.

```

function createSurvey(
    string memory _title,
    string memory _description,
    uint256 _durationInMinutes,
    string[] memory qTexts,
    string[] memory qTypes,
    string[][] memory qOptions
) public {
    Survey survey = new Survey(_title, _description, _durationInMinutes);

    for (uint i = 0; i < qTexts.length; i++) {
        survey.addQuestion(qTexts[i], qTypes[i], qOptions[i]);
    }

    allSurveys.push(address(survey));
    emit SurveyCreated(address(survey), _title);
}

function getAllSurveys() public view returns (address[] memory) {
    return allSurveys;
}

```

[그림 4-10] SurveyFactory.sol 일부 코드

또한, 본 시스템에서는 Web3.js 라이브러리를 활용하여 사용자의 MetaMask 지갑 연결과 계정 상태 복원 기능을 구현하였다. connectWallet 함수는 사용자가 직접 지갑 연결 버튼을 클릭했을 때 호출되고 window.ethereum의 존재 여부를 확인하여 MetaMask 설치 유무를 판단한다. 설치된 것을 확인하면 Web3 인스턴스를 생성한 뒤 ethereum.request({method:"eth_requestAccounts"})를 호출하여 사용자의 계정 접근 권한을 요청한다.

추가로 사용자의 지갑 연동을 완료한 상태에서 페이지를 새로고침 하거나 재방문 시 별도의 클릭 없이도 계정 정보를 자동으로 복원할 수 있도록 autoConnect 함수를 구현하였다. connectWallet과 autoConnect 로직을 결합함으로써, DApp 사용자는 최초 접속 시 지갑을 승인하여 연결할 수 있고, 이후 동일 브라우저 내에서는 추가적인 인증 절차 없이도 자동으로 연결된 계정 정보가 반영되어 원활한 서비스를 이용할 수 있다.

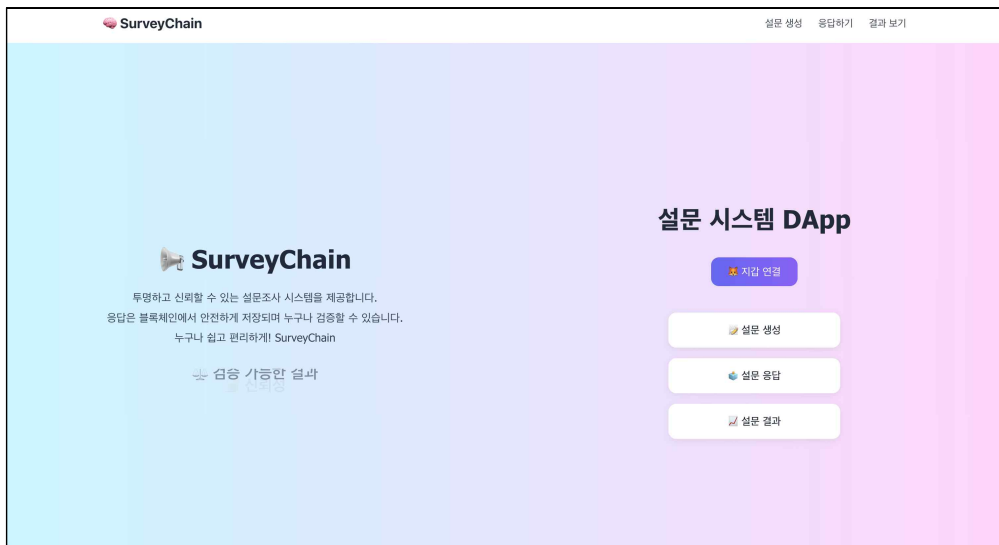
```
const connectWallet = async () => {
  if (window.ethereum) {
    const web3 = new Web3(window.ethereum);
    await window.ethereum.request({ method: "eth_requestAccounts" });
    const accounts = await web3.eth.getAccounts();
    setAccount(accounts[0]);
  } else {
    alert("메타마스크를 설치해주세요.");
  }
};

useEffect(() => {
  const autoConnect = async () => {
    if (window.ethereum && window.ethereum.selectedAddress) {
      const web3 = new Web3(window.ethereum);
      const accounts = await web3.eth.getAccounts();
      setAccount(accounts[0]);
    }
  };
  autoConnect();
}, []);
```

[그림 4-11] MetaMask 연동 및 지갑 연결 로직 코드

구현된 블록체인 기반 온라인 여론조사 시스템 SurveyChain의 메인페이지는 [그림 4-12]와 같다. 상단의 내비게이션 바를 통해 주요 기능을 제공하는 URL로 이동할 수 있다. 메인페이지는 좌측과 우측으로 나누어 구현하였으며, 좌측에는 시스템의 소개 글과 제공하는 이점을 언급하였고 우측에는 시스템의 기능을 바로 사용할 수 있도록 크게 시각화하여 배치했다. 각 버튼을 클릭하면 설문 생성, 설문 응답, 설문 결과 페이지로 이동되어 관련 기능을 사용할 수 있다.

[그림 4-13]은 설문 생성 버튼을 클릭했을 때 나오는 여론조사를 생성하는 페이지이다. 상단의 내비게이션 바는 고정하여 페이지별 편리한 이동을 가능하게 하였고, 설문 제목, 설문 설명, 마감일을 설정할 수 있다. 또한, 설문 문항을 생성할 수 있으며, [그림 4-14]와 같이 질문 추가 버튼을 통하여 설문 문항을 생성할 수 있다. 설문 문항은 객관식과 주관식으로 구성되어 있으며, 각 문항에 질문 내용을 남기고 선택지를 추가하여 여론조사를 생성할 수 있다.



[그림 4-12] 블록체인 기반 온라인 여론조사 시스템 메인페이지

설문 생성 응답하기 결과 보기

설문 생성

마감일:

질문

+ 질문 추가
✓ 설문 생성

🔗 지갑 주소: 0x80339C66Ee07F22DbAB41b0d8465bF458c126d01

[그림 4-13] 생성 페이지

질문

+ 선택지 추가

+ 질문 추가
✓ 설문 생성

[그림 4-14] 문항 생성 세부 화면

트랜잭션 요청

예상 변동 사항 ?

변경 사항 없음

요청자: ?

!

HTTP

localhost:3000

다음과 상호 작용: ?

?

Oxd1600...90642

네트워크 수수료 ?

0.0053

s

SepoliaETH

속도

추천 사이트

~12 초

취소

컨펌

[그림 4-15] 트랜잭션 요청 화면

localhost:3000 내용:

✓

설문이 생성되었습니다!

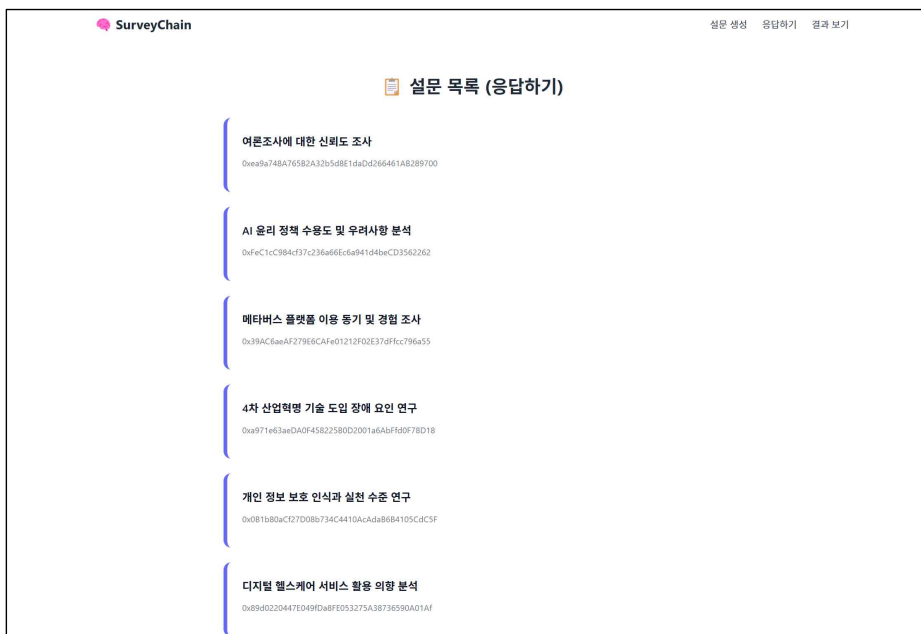
확인

[그림 4-16] 생성 완료 안내

[그림 4-15]는 설문 생성 버튼을 클릭하였을 때 나오는 MetaMask 트랜잭션 요청 안내 창이다. MetaMask는 요청자의 출처와 상호작용할 컨트랙트 주소, 지

불해야 할 네트워크 수수료(가스비), 예상 처리 시간 등을 보여주고 사용자의 서명을 기다린다. 사용자가 서명을 완료하면, 트랜잭션이 본 시스템의 구현 환경인 이더리움 세폴리아 테스트넷에 전파되어 설문 생성 로직이 실행된다. 또한, 설문 생성이 완료되면 [그림 4-16]과 같이 안내 창을 통해 편의성을 제공하였다.

응답하기 버튼을 클릭하면 [그림 4-17]에서와 같이 응답 가능한 설문 목록을 보여준다. MetaMask를 통해 이더리움 네트워크에 연결된 사용자가 현재까지 생성된 설문 목록을 조회할 수 있도록 구현되어 있으며, 페이지 상단에는 내비게이션 바를 통해 직관적인 접근을 돕는다. 중앙에는 여론조사 제목과 해당 여론조사를 생성한 스마트 컨트랙트 주소가 카드 형태로 나열되어 있으며, 각 항목을 클릭함으로써, 사용자는 응답 페이지로 이동할 수 있다. 각 여론조사는 이더리움 스마트 컨트랙트를 통해 독립적으로 배포되어 있다. 이러한 구조는 여론조사 데이터의 위·변조 방지 및 투명성을 보장하는 동시에, 사용자가 직접 참여한 여론조사를 확인할 수 있도록 돕는다. 시각적 구성은 여백 정렬을 활용하여 응답 목록을 깔끔히 표시하고 카드 형식의 디자인은 사용자 경험을 고려하였다.



[그림 4-17] 응답 가능 설문 목록 페이지


설문 생성 응답하기 결과 보기


설문: 여론조사에 대한 신뢰도 조사


설명: 본 조사는 여론조사에 대한 신뢰도 조사로, 근래에 많은 사람들이 여론조사의 위조 및 변조 가능성에 대해 많은 문제를 제기해왔습니다. 이에 대한 의견을 수렴해보고자 조사를 진행하오니 많은 참여 부탁드립니다.

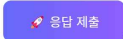

마감 시간: 2025. 5. 26. 오전 3:31:00

Q1: 여러분은 평소에 여론조사 결과를 얼마나 신뢰하시나요?

- ☐ 매우 신뢰한다
- ☐ 어느 정도 신뢰한다
- ☒ 보통이다
- ☐ 별로 신뢰하지 않는다
- ☐ 전혀 신뢰하지 않는다

Q2: 여론조사의 신뢰성을 높이기 위해 어떤 개선이 필요하다고 생각하시나요?

블록체인 기술을 도입한 무결성을 제공해야 한다고 생각합니다.



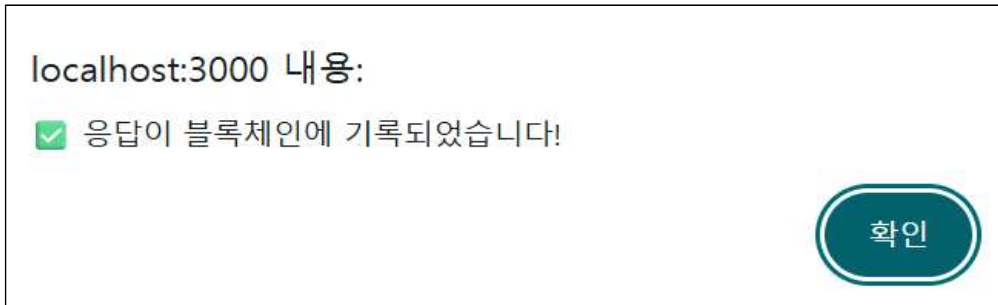

지갑 주소: 0x3326cFa853439735Ae309B3Ffc1b01e77bdA847A

[그림 4-18] 응답 페이지

여론조사 응답 목록 페이지에서 원하는 여론조사를 선택 후 들어가면 [그림 4-18]과 같이 응답 페이지를 확인할 수 있다. 해당 페이지에서는 생성한 여론조사에 대한 응답을 진행할 수 있으며, 생성된 여론조사에 따라 문항 차이가 존재하나, 객관식, 주관식 형식으로 응답을 진행할 수 있다. 또한, 해당 여론조사에 대한 설명과 마감 시간을 확인할 수 있다. 마감 시간이 되면 응답하기 페이지에서 해당 설문은 사라지며, 결과 보기 페이지에서 확인할 수 있다. 설문 문항이 객관식일 경우 radio 박스 형식으로 단일 선택 응답을 진행할 수 있으며, 주관식의 경우 텍스트로 입력을 진행할 수 있다. 하단에는 현재 연동되어 있는 본인의 MetaMask 주소를 확인할 수 있으며 응답 제출을 진행할 수 있다.

해당 절차는 먼저 Q1에서 체크한 객관식 선택지와 Q2에서 작성한 주관식 텍스트 입력을 수집 후, 사용자의 MetaMask 지갑 연동을 확인한다. 연동이 되어 있지 않다면 연결 요청을 진행하고 스마트 컨트랙트 함수를 호출하고 MetaMask

를



[그림 4-19] 응답 완료 안내

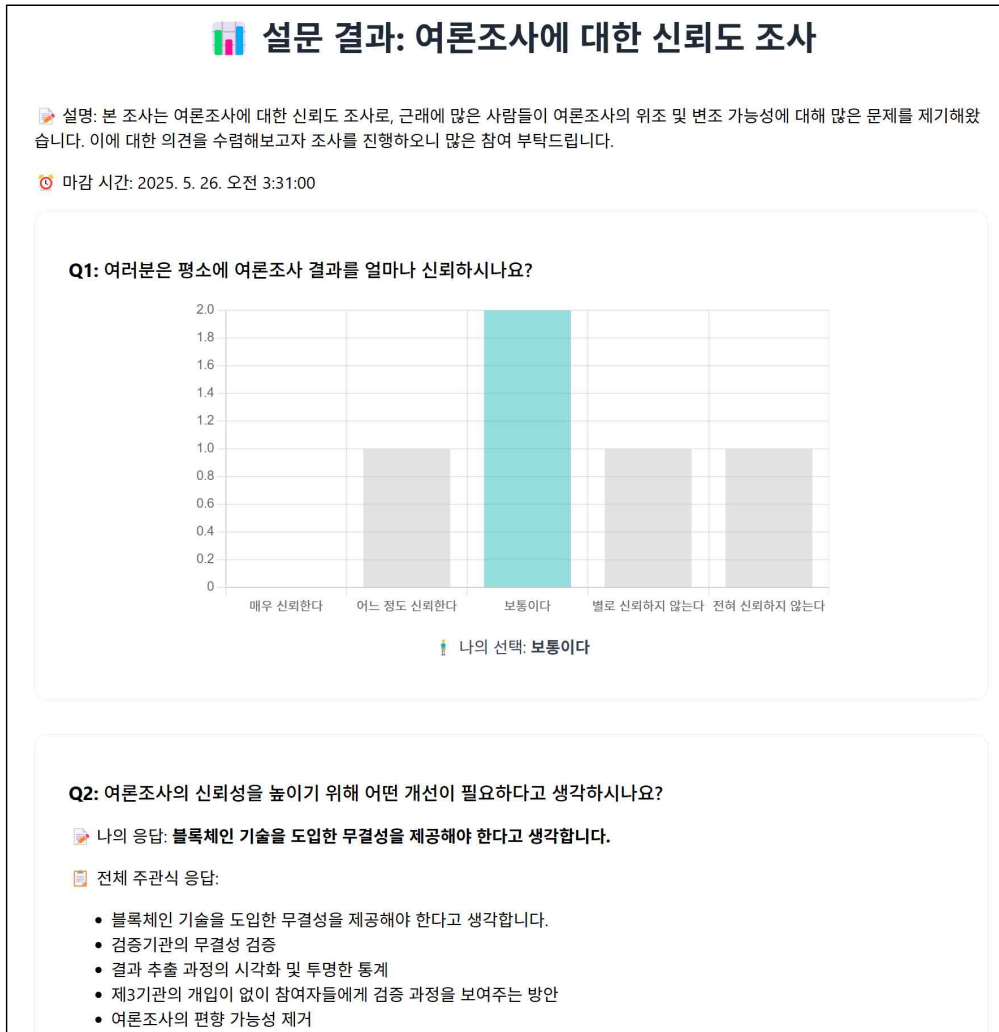


[그림 4-20] 결과 확인 페이지

통해 [그림 4-15]와 동일하게 트랜잭션 서명 요청을 진행한다. 사용자가 이를 수락하면 네트워크에 전송되며 블록체인에 변조 불가능한 형태로 올라가게 된다. 그 후, [그림 4-19]와 같이 “응답이 블록체인에 기록되었습니다!”라는 안내 창이 표시되며 응답이 정상적으로 올라간 것을 확인할 수 있다.

응답을 완료하고 마감 시간이 지나면 [그림 4-20]과 같이 결과 보기 페이지에서 확인할 수 있다. 그 후, [그림 4-21]과 같이 결과 페이지에 들어갈 수 있다. 해당 페이지는 사용자가 참여한 여론조사 결과를 시각화하여 확인할 수 있도록 구성된 결과 확인 화면으로, 응답 마감 이후 자동으로 집계된 데이터를 블록체인 네트워크에서 불러와 사용자에게 제공한다. 사용자의 응답은 사전에 정의된 스마트 컨트랙트를 통해 블록체인에 트랜잭션 형태로 기록되며, 이를 기반으로 집계

된 결과는 변조 불가능한 형태로 저장되기에 높은 신뢰도와 투명성을 보장한다.



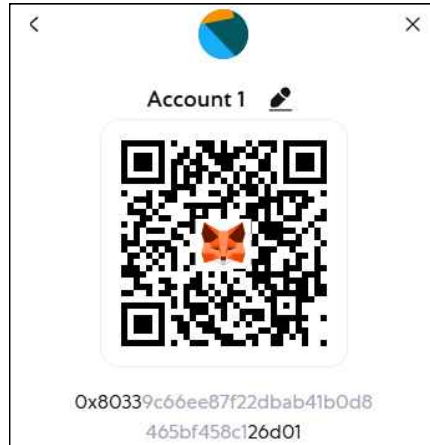
[그림 4-21] 결과 페이지

또한, 응답 데이터는 중앙 서버가 아닌 탈중앙화된 블록체인 네트워크에 저장되기 때문에, 설문 결과에 대한 조작 가능성을 원천적으로 차단하고 설문 참여자와 결과 열람자 모두 공정한 데이터 분석 결과를 제공할 수 있다. 이와 같은 구조는 기존 중앙 서버 기반의 온라인 여론조사 시스템이 가지는 데이터 위·변조, 관리자 개입 등의 문제점을 기술적으로 해결할 수 있다.

사용자는 해당 페이지에서 객관식 질문에 대한 전체 응답 분포를 그래프와 같이 시각적 요소를 통해 직관적으로 파악할 수 있으며, 주관식 응답의 경우 텍스트 형태로 응답을 확인할 수 있다. 특히 본인의 응답 반영 여부를 객관식의 경우 하이라이트를 통해 확인할 수 있으며, 주관식의 경우 ‘나의 응답’을 통해 본인이 작성한 내용을 확인할 수 있다. 이러한 결과 시각화 방식은 사용자의 이해도를 높이고 공공정책 수립, 여론 분석 등 다양한 활용 영역에 있어서 객관적이고 신뢰성 높은 데이터 기반 의사결정을 가능케 한다.

Transaction Hash:	0x045426b1cdf450384afc4ec5983951722bcedddbce97a54c633759f2fb69fec
Status:	Success
Block:	8414935 10969 Block Confirmations
Timestamp:	36 hrs ago (May-27-2025 03:39:24 AM UTC)
From:	0x80339C66Ee87F22DbAB41b0d8465bF458c126d01
To:	0x146E2cCA520eb4634A1e8813FC3eF46435DeC167 [0x541db78a80c7708f2005686590270658f40fed54 Created]
Value:	0 ETH
Transaction Fee:	0.005659637840121888 ETH
Gas Price:	2.500710424 Gwei (0.000000002500710424 ETH)

[그림 4-22] Etherscan 검증 화면



[그림 4-23] MetaMask 지갑 주소

또한, [그림 4-22]와 같이 Etherscan을 통하여 사용자는 트랜잭션 해시와 블록 번호, [그림 4-23]과 같이 참여자의 송·수신자 주소 등의 정보를 확인하고, 데이터의 무결성과 투명성, 거래 실행 여부 및 신뢰성을 검증할 수 있다. 이는 본 논문에서 제안하는 블록체인 기반 온라인 여론조사 시스템의 무결성, 투명성, 불변성을 보장하는 핵심 메커니즘이다.

결과적으로 해당 페이지는 블록체인 기반 온라인 여론조사 시스템의 핵심인 데이터 무결성과 투명성을 구현하는 기능으로, 탈중앙화된 여론조사 시스템의 신뢰성과 유효성을 사용자에게 체감적으로 전달하는 역할을 한다.

4.3 제안방식 분석

본 절에서는 제 3장에서 도출한 시스템 요구사항을 충족하기 위해 제안방식의 구체적인 동작 방식과 기대효과를 기술한다. 본 시스템은 블록체인 기반 온라인 여론조사 시스템으로, 설문 생성부터 응답 제출, 결과 공개에 이르기까지의 전 과정을 블록체인에 기록함으로써 데이터 위·변조 방지, 신뢰성 확보 등의 기능을 제공한다. 이에 대한 비교 분석은 <표 4-1>과 같다.

• 블록체인 기반 응답 기록을 통한 무결성 확보

사용자가 설문에 응답하면 해당 응답 내용은 스마트 컨트랙트를 통해 이더리움 블록체인에 트랜잭션 형태로 기록된다. 이는 탈중앙화된 네트워크의 합의를 통해 블록에 저장되므로, 관리자가 데이터를 수정하거나 삭제할 수 없는 구조를 보장한다. 이를 통해 사용자들은 여론조사 결과의 신뢰성을 확보할 수 있으며, 응답 데이터의 위·변조 우려를 해소할 수 있다.

- **실시간 집계 및 시각화를 통한 설문 결과 활용성 제고**

스마트 컨트랙트를 통해 응답 내역을 자동으로 집계하며, 프론트엔드에서는 이를 시각적으로 표현한다. 응답 분포는 그래프 형태로 시각화 방식으로 제공되며, 사용자는 실시간으로 결과를 확인할 수 있다. 이는 정책 결정, 여론 파악 등의 의사결정 과정에 유용하게 활용될 수 있다.

- **MetaMask 연동을 통한 사용자 인증 및 중복 응답 방지**

본 시스템은 MetaMask 지갑과 연동되어 사용자의 이더리움 세폴리아 테스트 넷 주소를 기반으로 응답을 수행한다. 한 주소당 한 번의 응답만 제출할 수 있도록 설계되어 중복 참여를 방지하며, 사용자의 익명성은 보장하면서도 신뢰할 수 있는 참여를 유도할 수 있다.

<표 4-1> 제안 아이템 비교 분석표

	서울시 여론조사	청주시선	진재환 et al. 블록체인 시스템 (2018)	제안 아이템
시스템 구조	중앙 서버 기반	중앙 서버 기반	프라이빗 이더리움	퍼블릭 이더리움
참여자 인증·검증	휴대 전화 본인 인증	휴대 전화 본인 인증	이메일 인증	MetaMask 연동을 통한 응답 제한
응답 무결성	중앙 서버 분석으로 조작·변조 가능	중앙 서버 분석으로 조작·변조 가능	암호화 및 복호화 키로 검증 가능	Etherscan을 통해 무결성 검증
개별 응답 반영 투명성	반영 여부 확인 불가	반영 여부 표시 (투명성 제한)	전체 재집계 결과 비교 검증	결과 페이지 및 Etherscan으로 확인 가능
결과 시각화 제공 형태	기본 통계 차트 제공	리포트 형태	검증 화면 제공 (차트화 미제공)	차트·응답 하이라이트
상태 관리	미제공	미제공	단계별 제어로 인한 가스비 발생	설문 생성부터 마감까지 자동 관리
확장성· 유지보수	행정 시스템 연동 필요	콘텐츠 증대 시 관리 부담	프라이빗 네트워크 및 스마트 컨트랙트 업그레이드 필요	React 모듈화 및 세폴리아 네트워크 활용으로 손쉬운 기능 추가·배포
수수료 부담	예측 가능한 중앙 서버 운영 예산	예측 가능한 중앙 서버 운영 예산	퍼블릭 대비 낮은 수수료	퍼블릭 네트워크 상의 가스 수수료가 높고 큰 변동 폭
처리 지연	중앙 서버 처리로 빠른 속도	중앙 서버 처리로 빠른 속도	프라이빗 네트워크의 상대적 빠른 생성 주기	퍼블릭 네트워크 특성상 블록 생성 지연

제 5 장 결론 및 향후 계획

본 논문에서는 이더리움 세폴리아 테스트넷 기반의 블록체인 온라인 여론조사 DApp을 설계 및 구현하여 [그림 5-1]과 같은 기대효과를 가지고 중앙집중형 시스템의 조작 및 삭제 위험을 제거하고 데이터 무결성과 투명성을 확보하였다. 여론조사 생성부터 응답 집계까지의 전 과정을 스마트 컨트랙트에 기록함으로써 관리자의 임의 변경을 원천 차단하였으며, MetaMask 지갑 연동을 통해 참여자의 계정 주소를 신원으로 활용하고 한 계정당 한 번만 응답할 수 있도록 하여 중복 응답을 방지하였다.

설문 마감 후 응답 현황을 막대 그래프로 시각화하여 사용자가 자신의 응답을 하이라이트 표시로 직관적으로 확인할 수 있도록 하였다. 또한, 마감된 설문은 추가 응답이 차단되고 바로 결과 확인으로 전환되어 사용자에게 명확한 참여 흐름을 제공한다. SurveyFactory 컨트랙트의 SurveyCreated 이벤트를 감지하여 설문을 생성한 직후 자동 리다이렉션을 구현함으로써, 설문 생성부터 참여 결과 조회까지의 매끄러운 UX(User experience)를 보장한다. 나아가 React.js와 Web3.js 기반의 모듈화된 구조는 기능 확장과 테스트넷에서 메인넷으로의 전환을 용이하게 만들어, 시스템의 유지보수 가능성을 높였다.



[그림 5-1] 기대효과

향후에는 사용성 및 운영 효율성을 강화하기 위해 UX/UI 디자인 리뉴얼을 진행할 계획이다. 이를 통해 반응형 대시보드와 참여율 그래프 등 시각화 요소를 도입할 예정이다. 또한, 영지식증명 기반 익명 응답 기능과 지갑 주소 및 응답 내용 분리 저장 구조를 연구하고 적용하여 프라이버시를 강화할 예정이다. 마지막으로, 실시간 알림 기능과 자유 응답 등 세부 분석 대시보드를 개발함으로써 운영자와 참여자 모두에게 즉각적인 인사이트를 제공하고자 한다.

마지막으로, 스마트 컨트랙트 보안 이슈 해결과 이더리움 세폴리아 테스트넷에서 메인넷으로 마이그레이션을 준비함과 동시에, 사용자 참여를 촉진하기 위해 여론조사 참여 독려를 위한 수집형 게임화 기능을 포함한 NFT(Non Fungible Token) 기반 보상 모델과 다국어 지원 시스템을 도입하여 글로벌 확장 가능성을 확보한다. 이를 통해 실용성과 보안성 모두를 갖춘 완전한 블록체인 기반 온라인 여론조사 시스템으로 발전시킬 것이다.

[참 고 문 헌]

- [1] 여론 속의 여론, <https://hrcopinion.co.kr/archives/27300>, 2023
- [2] 성기정, 정채린, 조은아, 이종호, 김희영, 김영우, 이경현. (2018). 블록체인 기반 교내 전자투표 시스템. 정보보호학회논문지, 28(4), 779-787.
- [3] 정일권 and 김영석. (2009). 온라인 여론조사에 대한 수용자의 평가와 영향력에 관한 연구. 한국방송학보, 23(3), 247-285.
- [4] 맹보학, 안문석, 박성진. (2002). 인터넷을 이용한 설문조사방법. 정부학연구, 8(1), 9-308.
- [5] 서울시 여론조사, “<https://research.seoul.go.kr/search/main.do>”, 2023
- [6] 청주시선, “<https://siseon.cheongju.go.kr/homepage/main>”, 2020
- [7] 진재환, 엄현민, 선주은 and 이명준. (2018). 견고한 검증을 제공하는 이더리움 블록체인 기반의 여론조사 어플리케이션. 예술인문사회 융합 멀티미디어 논문지, 8(3), 895-905.
- [8] React.js, “<https://ko.legacy.reactjs.org/>”, 2025
- [9] HTML, “<https://developer.mozilla.org/ko/docs/Web/HTML>”, 2025
- [10] CSS, “<https://developer.mozilla.org/ko/docs/Web/CSS>”, 2025
- [11] JavaScript, “<https://developer.mozilla.org/ko/docs/Web/JavaScript>”, 2025
- [12] Web3.js, “<https://web3js.readthedocs.io/en/v1.10.0/>”, 2016

- [13] Wensheng Gan, Zhenqiang Ye, Shicheng Wan, and Philip S. Yu. 2023. Web 3.0: The Future of Internet. In Companion Proceedings of the ACM Web Conference 2023 (WWW '23 Companion). Association for Computing Machinery, New York, NY, USA, 1266–1275.
- [14] MetaMask, “<https://metamask.io/>”, 2025
- [15] Infura, “<https://www.infura.io/>” 2025
- [16] Solidity, “<https://docs.soliditylang.org/en/v0.8.30/>”, 2025
- [17] 김철진. (2018). 블록체인 기반의 스마트 컨트랙트 정적/동적 설계 기법. 한국산학기술학회논문지, 19(6), 110–119.
- [18] Truffle, “<https://archive.trufflesuite.com/>”, 2022
- [19] 김신, 오가인, 정경량, 김효나, 박성혜, 박철훈. (2019). 태양광 에너지를 활용한 블록체인 기반 P2P 이웃간 전력 거래 설계. 한국정보처리학회 학술대회논문집, 26(2), 574–577.
- [20] Visual Studio Code, “<https://code.visualstudio.com/>”, 2025
- [21] Etherscan, “<https://etherscan.io/>”, 2025

[부 록] 온라인 여론조사의 조작 방지를 위한 블록체인 기반 시스템 설계 및 구현

Survey.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

contract Survey {
    address public owner;
    string public title;
    string public description;
    uint256 public deadline;

    struct Question {
        string text;
        string qType;      // "text" 또는 "multiple"
        string[] options;  // 객관식일 경우에만 사용
    }

    Question[] public questions;

    // 응답 여부 및 원본 응답 저장
    mapping(address => bool) public hasVoted;
    mapping(address => string[]) private responses;

    // [추가] 객관식 문항별 보기 옵션 집계
    // optionCounts[questionIndex][optionIndex] = 투표 수
    mapping(uint256 => mapping(uint256 => uint256)) public optionCounts;

    // [추가] 주관식(텍스트) 문항별 모든 응답 저장
    mapping(uint256 => string[]) public subjectiveAnswers;

    constructor(
        string memory _title,
        string memory _description,
        uint256 _durationInMinutes
    ) {
        owner = msg.sender;
        title = _title;
        description = _description;
        deadline = block.timestamp + (_durationInMinutes * 1 minutes);
    }
}
```

```

// 설문에 질문 추가 (only owner)
function addQuestion(
    string memory _text,
    string memory _qType,
    string[] memory _options
) public {
    require(msg.sender == owner, "Only owner can add questions.");

    questions.push(
        Question({
            text: _text,
            qType: _qType,
            options: _options
        })
    );
}

// 설문 마감 여부 확인
function isClosed() public view returns (bool) {
    return block.timestamp >= deadline;
}

// 질문 수 반환
function getQuestionCount() public view returns (uint256) {
    return questions.length;
}

// 질문 1개 반환 (text, qType, options)
function getQuestion(uint256 index) public view returns (
    string memory, string memory, string[] memory
) {
    Question storage q = questions[index];
    return (q.text, q.qType, q.options);
}

// 설문 응답 제출 (한 계정 1회, 기간 내에만 가능)
function submitResponse(string[] memory _responses) public {
    require(!hasVoted[msg.sender], "Already responded.");
    require(block.timestamp < deadline, "Survey closed.");
    require(_responses.length == questions.length, "Invalid response count.");

    hasVoted[msg.sender] = true;
    responses[msg.sender] = _responses;
}

```

```

    for (uint256 i = 0; i < questions.length; i++) {
        bytes32 qTypeHash = keccak256(bytes(questions[i].qType));
        bytes32 multipleHash = keccak256(bytes("multiple"));
        bytes32 textHash = keccak256(bytes("text"));

        if (qTypeHash == multipleHash) {
            // 객관식 처리
            for (uint256 j = 0; j < questions[i].options.length; j++) {
                if (keccak256(bytes(_responses[i])) == keccak256(bytes(questions[i].options[j]))) {
                    optionCounts[i][j] += 1;
                    break;
                }
            }
        } else if (qTypeHash == textHash) {
            // 주관식(텍스트) 응답 저장
            subjectiveAnswers[i].push(_responses[i]);
        }
    }

    // 특정 사용자의 원본 응답 조회
    function getResponse(address user) public view returns (string[] memory) {
        return responses[user];
    }

    // 객관식 문항별 집계 결과 조회
    function getResults(uint256 questionIndex) public view returns (uint256[] memory) {
        require(questionIndex < questions.length, "Invalid question index");
        require(
            keccak256(bytes(questions[questionIndex].qType)) == keccak256(bytes("multiple")),
            "Not a multiple-choice question"
        );

        uint256 len = questions[questionIndex].options.length;
        uint256[] memory counts = new uint256[](len);
        for (uint256 i = 0; i < len; i++) {
            counts[i] = optionCounts[questionIndex][i];
        }
        return counts;
    }
}

```

```

// 주관식(텍스트) 문항별 모든 응답 조회
function getSubjectiveAnswers(uint256 questionIndex) public view returns (string[] memory) {
    require(questionIndex < questions.length, "Invalid question index");
    require(
        keccak256(bytes(questions[questionIndex].qType)) == keccak256(bytes("text")),
        "Not a text question"
    );
    return subjectiveAnswers[questionIndex];
}
}

```

SurveyFactory.sol

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

import "./Survey.sol";

contract SurveyFactory {
    address[] public allSurveys;

    struct QuestionInput {
        string text;
        string qType;
        string[] options;
    }

    event SurveyCreated(address surveyAddress, string title);

    function createSurvey(
        string memory _title,
        string memory _description,
        uint256 _durationInMinutes,
        string[] memory qTexts,
        string[] memory qTypes,
        string[][] memory qOptions
    ) public {
        Survey survey = new Survey(_title, _description, _durationInMinutes);

        for (uint i = 0; i < qTexts.length; i++) {
            survey.addQuestion(qTexts[i], qTypes[i], qOptions[i]);
        }
    }
}

```

```

    }

    allSurveys.push(address(survey));
    emit SurveyCreated(address(survey), _title);
  }

  function getAllSurveys() public view returns (address[] memory) {
    return allSurveys;
  }
}

```

CreateSurvey.js

```

import React, { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";
import Web3 from "web3";
import SurveyFactoryJSON from "../contract/SurveyFactoryABI.json";
import "../CreateSurvey.css";

export default function CreateSurvey() {
  const [title, setTitle] = useState("");
  const [description, setDescription] = useState("");
  const [questions, setQuestions] = useState([]);
  const [deadline, setDeadline] = useState("");

  const [web3, setWeb3] = useState(null);
  const [factory, setFactory] = useState(null);
  const [account, setAccount] = useState("");

  const navigate = useNavigate();

  useEffect(() => {
    const initWeb3 = async () => {
      if (window.ethereum) {
        const web3Instance = new Web3(window.ethereum);
        await window.ethereum.request({ method: "eth_requestAccounts" });
        const accounts = await web3Instance.eth.getAccounts();

        const factoryInstance = new web3Instance.eth.Contract(
          SurveyFactoryJSON.abi,
          SurveyFactoryJSON.address
        );
      }
    };
  });

```



```

        setWeb3(web3Instance);
        setAccount(accounts[0]);
        setFactory(factoryInstance);
    } else {
        alert("MetaMask를 설치해주세요!");
    }
};
initWeb3();
}, []);

const addQuestion = () => {
    setQuestions([
        ...questions,
        {
            text: "",
            type: "text",
            options: [""],
        },
    ]);
};

const handleQuestionChange = (index, key, value) => {
    const newQuestions = [...questions];
    newQuestions[index][key] = value;
    setQuestions(newQuestions);
};

const handleOptionChange = (qIndex, oIndex, value) => {
    const newQuestions = [...questions];
    newQuestions[qIndex].options[oIndex] = value;
    setQuestions(newQuestions);
};

const addOption = (index) => {
    const newQuestions = [...questions];
    newQuestions[index].options.push("");
    setQuestions(newQuestions);
};

const handleSubmit = async () => {
    if (!web3 || !factory || !account) {
        alert("Web3 연결 오류");
        return;
    }
};

```

```

}

const qTexts = questions.map((q) => q.text);
const qTypes = questions.map((q) => q.type);
const qOptions = questions.map((q) => (q.type === "multiple" ? q.options : []));

const duration = Math.ceil((new Date(deadline).getTime() - Date.now()) / 60000);

try {
  const tx = await factory.methods
    .createSurvey(title, description, duration, qTexts, qTypes, qOptions)
    .send({ from: account });

  const event = tx.events?.SurveyCreated;
  const surveyAddress = event?.returnValues?.surveyAddress;

  if (surveyAddress) {
    alert("설문이 생성되었습니다!");
    console.log("생성된 설문 주소:", surveyAddress);
    navigate(`/respond`);
  } else {
    alert("설문 생성은 완료되었지만 주소를 불러오지 못했습니다.");
    console.log("전체 이벤트 로그:", tx.events);
  }
} catch (err) {
  console.error("설문 생성 중 오류:", err);
  alert("설문 생성 중 오류가 발생하였습니다.");
}

};

return (
  <>
    { /* 네비게이션 바 추가 */}
    <nav className="navbar">
      <div className="navbar-logo"><span onClick={() => navigate("/")}>SurveyChain</span></div>
      <div className="navbar-links">
        <span onClick={() => navigate("/create")}>설문 생성</span>
        <span onClick={() => navigate("/respond")}>응답하기</span>
        <span onClick={() => navigate("/results")}>결과 보기</span>
      </div>
    </nav>
  </>
);

```

```

{/*   설문 생성 UI */}
<div className="create-container">
  <h2 className="create-title">설문 생성</h2>
  <input
    className="input"
    placeholder="설문 제목"
    value={title}
    onChange={(e) => setTitle(e.target.value)}
  />
  <textarea
    className="textarea"
    placeholder="설문 설명"
    value={description}
    onChange={(e) => setDescription(e.target.value)}
  />

  <label>마감일:</label>
  <input
    type="datetime-local"
    className="datetime"
    value={deadline}
    onChange={(e) => setDeadline(e.target.value)}
  />

  <h3>질문</h3>
  {questions.map((q, idx) => (
    <div key={idx} className="question-card">
      <input
        className="input"
        placeholder="질문 내용"
        value={q.text}
        onChange={(e) => handleQuestionChange(idx, "text", e.target.value)}
      />
      <select
        className="select"
        value={q.type}
        onChange={(e) => handleQuestionChange(idx, "type", e.target.value)}
      >
        <option value="text">주관식 (텍스트)</option>
        <option value="multiple">객관식 (선택형)</option>
      </select>

      {q.type === "multiple" && (

```

```

        <div>
          {q.options.map((opt, oidx) => (
            <input
              key={oidx}
              className="option-input"
              placeholder={`선택지 ${oidx + 1}`}
              value={opt}
              onChange={(e) => handleOptionChange(idx, oidx, e.target.value)}
            />
          ))}
          <button onClick={() => addOption(idx)} className="button"> 선택지 추가
        </button>
      </div>
    )}
  </div>
  )))

  <button onClick={addQuestion} className="button"> 질문 추가</button>
  <button onClick={handleSubmit} className="button"> 설문 생성</button>

  {account && (
    <p className="wallet"> 지갑 주소: {account}</p>
  )}
</div>
</>
);
}

```

PollPage.js

```

import React, { useEffect, useState } from "react";
import { useParams, useNavigate } from "react-router-dom";
import Web3 from "web3";
import SurveyABI from "../contract/SurveyABI.json";
import "../PollPage.css";

export default function PollPage() {
  const { id } = useParams();
  const [web3, setWeb3] = useState(null);
  const [survey, setSurvey] = useState(null);
  const [account, setAccount] = useState("");
  const [title, setTitle] = useState("");
  const [description, setDescription] = useState("");

```

```

const [deadline, setDeadline] = useState("");
const [questions, setQuestions] = useState([]);
const [answers, setAnswers] = useState([]);
const navigate = useNavigate();

// 마감 시간 포매팅
const formattedDeadline = deadline
  ? new Date(Number(deadline) * 1000).toLocaleString()
  : "";

useEffect(() => {
  const loadSurvey = async () => {
    if (window.ethereum) {
      const web3Instance = new Web3(window.ethereum);
      await window.ethereum.request({ method: "eth_requestAccounts" });
      const accounts = await web3Instance.eth.getAccounts();

      const surveyInstance = new web3Instance.eth.Contract(SurveyABI, id);
      const fetchedTitle = await surveyInstance.methods.title().call();
      const fetchedDescription = await surveyInstance.methods.description().call();
      const fetchedDeadline = await surveyInstance.methods.deadline().call();
      const questionCount = await surveyInstance.methods.getQuestionCount().call();

      const loadedQuestions = [];
      for (let i = 0; i < questionCount; i++) {
        const q = await surveyInstance.methods.getQuestion(i).call();
        loadedQuestions.push({ text: q[0], qType: q[1], options: q[2] });
      }

      setWeb3(web3Instance);
      setAccount(accounts[0]);
      setSurvey(surveyInstance);
      setTitle(fetchedTitle);
      setDescription(fetchedDescription);
      setDeadline(fetchedDeadline);
      setQuestions(loadedQuestions);
      setAnswers(new Array(questionCount).fill(""));
    } else {
      alert("MetaMask를 설치해주세요!");
    }
  };

  loadSurvey();

```

```

    }, [id]));

const handleAnswerChange = (index, value) => {
  const newAnswers = [...answers];
  newAnswers[index] = value;
  setAnswers(newAnswers);
};

const handleSubmitAnswers = async () => {
  if (!survey || !account) return alert("Web3 연결이 안 되어 있어요!");

  try {
    await survey.methods.submitResponse(answers).send({ from: account });
    alert("    응답이 블록체인에 기록되었습니다!");
  } catch (err) {
    console.error("    응답 제출 실패:", err);
    alert("응답 제출 중 오류가 발생했습니다.");
  }
};

return (
  <>
    { /* 네비게이션 바 */ }
    <nav className="navbar">
      <div className="navbar-logo"><span onClick={() => navigate("/")}>SurveyChain</span></div>
      <div className="navbar-links">
        <span onClick={() => navigate("/create")}>설문 생성</span>
        <span onClick={() => navigate("/respond")}>응답하기</span>
        <span onClick={() => navigate("/results")}>결과 보기</span>
      </div>
    </nav>

    <div className="poll-container">
      <h2 className="poll-title">설문: {title}</h2>
      <p className="poll-description">설명: {description}</p>
      <p className="poll-deadline">마감 시간: {formattedDeadline}</p>

      {questions.length > 0 ? (
        questions.map((q, idx) => (
          <div key={idx} className="poll-question-block">
            <p className="poll-question-text">
              <strong>Q{idx + 1}</strong> {q.text}

```

```

</p>

{q.qType === "text" ? (
  <input
    type="text"
    className="poll-input"
    value={answers[idx] ?? ""}
    onChange={(e) => handleAnswerChange(idx, e.target.value)}
  />
) : Array.isArray(q.options) && q.options.length > 0 ? (
  q.options.map((opt, oidx) => (
    <div key={oidx} className="poll-option">
      <label>
        <input
          type="radio"
          name={`question-${idx}`}
          value={opt}
          checked={answers[idx] === opt}
          onChange={() => handleAnswerChange(idx, opt)}
        />
        {opt}
      </label>
    </div>
  ))
) : (
  <p className="poll-warning"> 객관식 옵션이 없습니다.</p>
)}
</div>

))
) : (
  <p>설문을 불러오는 중입니다...</p>
)}

<button className="poll-submit-button" onClick={handleSubmitAnswers}>
  응답 제출
</button>

{account && <p className="poll-wallet">지갑 주소: {account}</p>}
</div>
</>
);
}

```

ResultPage.js

```
import React, { useEffect, useState } from "react";
import { useParams, useNavigate } from "react-router-dom";
import Web3 from "web3";
import SurveyABI from "../contract/SurveyABI.json";
import axios from "axios";
import { Bar } from "react-chartjs-2";
import { Chart, BarElement, CategoryScale, LinearScale } from "chart.js";
import "../ResultPage.css";

Chart.register(BarElement, CategoryScale, LinearScale);

// Etherscan API 설정
const ETHERSCAN_API_KEY = process.env.REACT_APP_ETHERSCAN_API_KEY;
const ETHERSCAN_BASE_URL = 'https://api-sepolia.etherscan.io/api';

// 내 계정이 설문 컨트랙트에 보낸 트랜잭션 해시만 조회
async function fetchMySurveyTx(account, surveyAddress) {
  const params = {
    module: 'account',
    action: 'txlist',
    address: account,
    startblock: 0,
    endblock: 'latest',
    sort: 'asc',
    apikey: ETHERSCAN_API_KEY,
  };
  const { data } = await axios.get(ETHERSCAN_BASE_URL, { params });
  return data.result
    .filter(tx => tx.to && tx.to.toLowerCase() === surveyAddress.toLowerCase())
    .map(tx => tx.hash);
}

// 트랜잭션 영수증 조회
async function fetchTxReceipt(txHash) {
  const params = {
    module: 'proxy',
    action: 'eth_getTransactionReceipt',
    txhash: txHash,
    apikey: ETHERSCAN_API_KEY,
  };
  const { data } = await axios.get(ETHERSCAN_BASE_URL, { params });
  return data.result;
```



```

}

export default function ResultPage() {
  const { id } = useParams(); // 설문 컨트랙트 주소
  const navigate = useNavigate();

  const [web3, setWeb3] = useState(null);
  const [survey, setSurvey] = useState(null);
  const [account, setAccount] = useState("");
  const [title, setTitle] = useState("");
  const [description, setDescription] = useState(""); // 설문 설명
  const [deadline, setDeadline] = useState(""); // 설문 마감 시간 (timestamp)
  const [questions, setQuestions] = useState([]);
  const [myAnswers, setMyAnswers] = useState([]);
  const [responseCounts, setResponseCounts] = useState([]);
  const [subjectiveAnswersAll, setSubjectiveAnswersAll] = useState([]); // 전체 주관식 응
  답

  // 무결성 검증 state
  const [txHashes, setTxHashes] = useState([]);
  const [receipts, setReceipts] = useState({});

  // load survey data and account
  useEffect(() => {
    const loadResults = async () => {
      if (!window.ethereum) {
        alert("MetaMask를 설치해주세요!");
        return;
      }
      const web3Instance = new Web3(window.ethereum);
      await window.ethereum.request({ method: "eth_requestAccounts" });
      const accounts = await web3Instance.eth.getAccounts();
      const acc = accounts[0];
      setWeb3(web3Instance);
      setAccount(acc);

      const surveyInstance = new web3Instance.eth.Contract(SurveyABI, id);
      setSurvey(surveyInstance);

      // 설문 제목, 설명, 마감 시간, 질문 가져오기
      const fetchedTitle = await surveyInstance.methods.title().call();
      const fetchedDescription = await surveyInstance.methods.description().call();
      const fetchedDeadline = await surveyInstance.methods.deadline().call(); // UNIX tim

```

```

estamp (seconds)

    const questionCount = await surveyInstance.methods.getQuestionCount().call();

    setTitle(fetchedTitle);
    setDescription(fetchedDescription);
    setDeadline(fetchedDeadline);

    const qList = [];
    for (let i = 0; i < questionCount; i++) {
        const q = await surveyInstance.methods.getQuestion(i).call();
        qList.push({ text: q[0], qType: q[1], options: q[2] });
    }
    setQuestions(qList);

    // 나의 응답
    const myResp = await surveyInstance.methods.getResponse(acc).call();
    setMyAnswers(myResp);

    // 전체 응답 집계 및 주관식 응답
    const counts = [];
    const textsAll = [];
    for (let i = 0; i < qList.length; i++) {
        if (qList[i].qType === "multiple") {
            const resultArr = await surveyInstance.methods.getResults(i).call();
            counts.push(resultArr.map(c => Number(c)));
            textsAll.push(null);
        } else {
            const allText = await surveyInstance.methods.getSubjectiveAnswers(i).call();
            counts.push(null);
            textsAll.push(allText);
        }
    }
    setResponseCounts(counts);
    setSubjectiveAnswersAll(textsAll);
};

loadResults();

// MetaMask 계정 변경 감지
if (window.ethereum) {
    window.ethereum.on('accountsChanged', accs => setAccount(accs[0] || ""));
}
}, [id]);

```

```

// 내 트랜잭션 조회
useEffect(() => {
  if (!account) return;
  fetchMySurveyTxS(account, id)
    .then(hashes => setTxHashes(hashes))
    .catch(console.error);
}, [account, id]);

// 영수증 조회
useEffect(() => {
  txHashes.forEach(txHash => {
    if (!receipts[txHash]) {
      fetchTxReceipt(txHash)
        .then(receipt => setReceipts(prev => ({ ...prev, [txHash]: receipt })))
        .catch(console.error);
    }
  });
}, [txHashes, receipts]);

// 마감 시간 포매팅
const formattedDeadline = deadline
  ? new Date(Number(deadline) * 1000).toLocaleString()
  : "";

return (
  <>
    { /* 네비게이션 바 */ }
    <nav className="navbar">
      <div className="navbar-logo"><span onClick={() => navigate("/")}>SurveyChain</span></div>
      <div className="navbar-links">
        <span onClick={() => navigate("/create")}>설문 생성</span>
        <span onClick={() => navigate("/respond")}>응답하기</span>
        <span onClick={() => navigate("/results")}>결과 보기</span>
      </div>
    </nav>

    <div className="result-page-container">
      <h2 className="result-page-title">설문 결과: {title}</h2>
      <p className="survey-description">설명: {description}</p>
      <p className="survey-deadline">마감 시간: {formattedDeadline}</p>

      {questions.map((q, idx) => (

```

```

<div key={idx} className="question-result-block">
  <p className="question-text">
    <strong>Q{idx + 1}</strong> {q.text}
  </p>

  {q.qType === "text" ? (
    <div className="subjective-list">
      <p>나의 응답: <strong>{myAnswers[idx] || "없음"}</strong></p>
      <p>전체 주관식 응답:</p>
      <ul>
        {subjectiveAnswersAll[idx] && subjectiveAnswersAll[idx].length > 0 ? (
          subjectiveAnswersAll[idx].map((ans, j) => <li key={j}>{ans}</li>)
        ) : (
          <li>아직 응답이 없습니다.</li>
        )}
      </ul>
    </div>
  ) : (
    <>
      <div className="chart-wrapper">
        <Bar
          data={{
            labels: q.options,
            datasets: [
              {
                label: "응답 수",
                data: responseCounts[idx] || [],
                backgroundColor: q.options.map((opt) =>
                  myAnswers[idx] === opt
                    ? "rgba(75, 192, 192, 0.6)"
                    : "rgba(200, 200, 200, 0.6)"
                ),
              },
            ],
          }}
          options={{ responsive: true, plugins: { legend: { display: false } } }}
        />
      </div>
      <p className="my-answer">
        나의 선택: <strong>{myAnswers[idx]}</strong>
      </p>
    </>
  )}

```

```

    })
  </div>
  )})

  { /* 무결성 검증 섹션 */ }
  <div className="integrity-section">
    <h3>응답 무결성 검증</h3>
    { txHashes.length === 0 ? (
      <p>응답된 트랜잭션이 없습니다. </p>
    ) : (
      <ul className="tx-list">
        { txHashes.map(txHash => {
          const r = receipts[txHash];
          const confirmed = r && r.blockNumber;
          const blockNum = confirmed ? parseInt(r.blockNumber, 16) : null;
          const status = r && r.status === '0x1' ? '성공' : '실패';
          return (
            <li key={txHash} className="tx-item">
              <a href={`https://sepolia.etherscan.io/tx/${txHash}`} target="_blank"
rel="noopener noreferrer">
                {txHash}
              </a>
              {confirmed ? (
                <div className="tx-detail">
                  <span>블록: {blockNum}</span>
                  <span>상태: {status}</span>
                  <span>가스: {parseInt(r.gasUsed, 16)}</span>
                </div>
              ) : (
                <span>조회 중...</span>
              )}
            </li>
          );
        })}
      </ul>
    )}
  </div>
</div>
</>
);
}

```